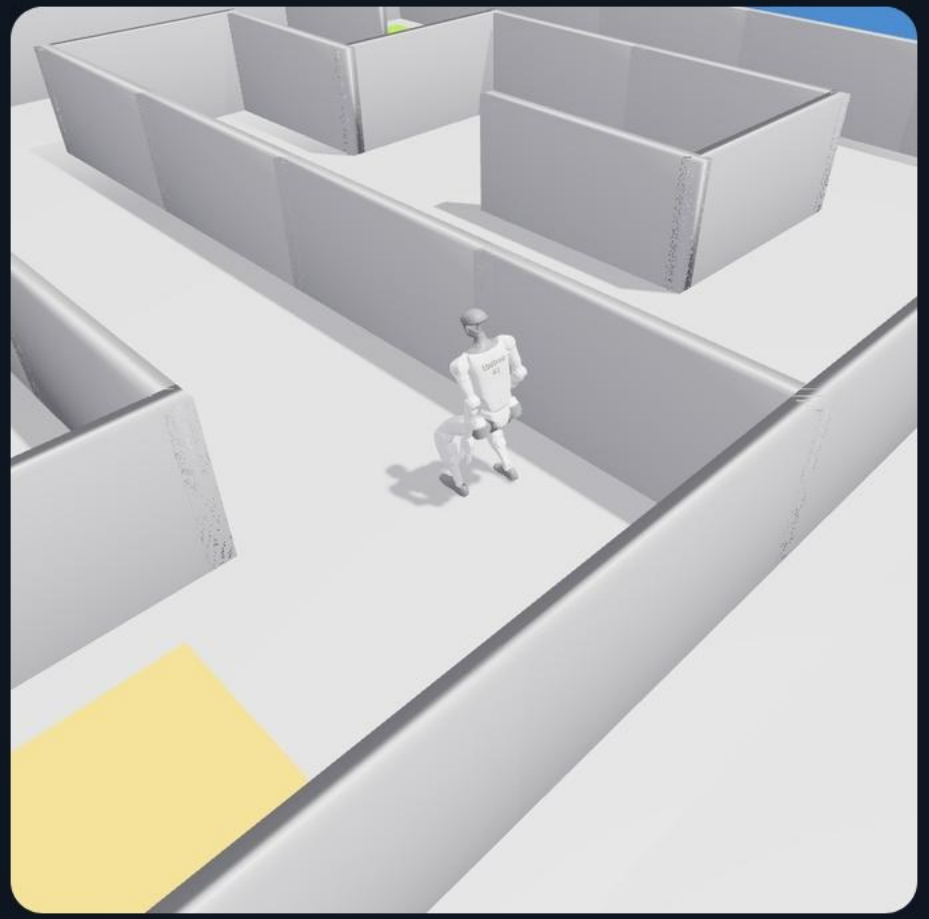
ENGINEERING WORKSHOP · ISAAC LAB 3 + NEWTON

Simulation for Humanoid

Week 7 — Curriculum learning · G1 Maze Challenge

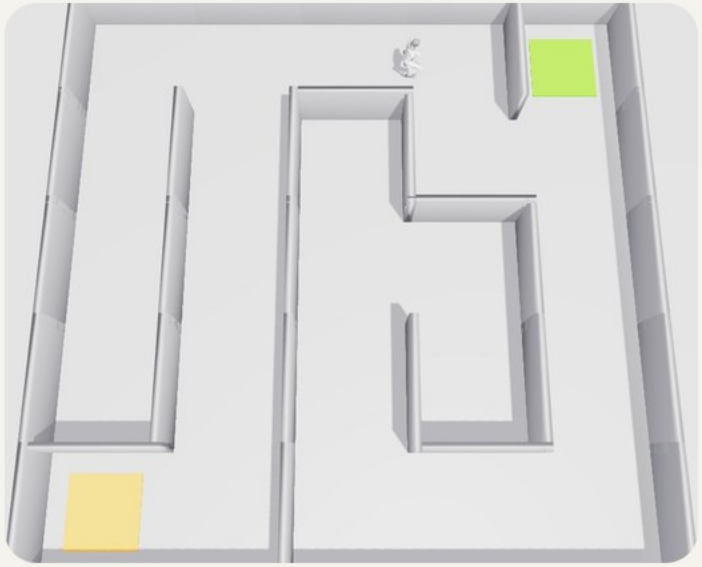
Isaac Lab 3.0 + Newton 2026 edition

Pannatorn Bongkotamphon
Dr. Sukhum Sattaratnamai



WEEK 7

Agenda



01 Recap + เป้าหมายวันนี้

02 Curriculum learning คืออะไร

03 Curriculum ใน Isaac Lab 3

04 Lab 7.1 + ตัวอย่างเพิ่มเติม

05 G1 Maze Challenge · height scanner

06 Exercises

RECAP · WEEK 3 – 6

G1 เดินตามคำสั่งได้แล้ว แต่ยังต้องมีคนกดปุ่มอยู่

Week 3 · play_interactive.py

ขับ G1 ด้วยคีย์บอร์ด

IJKL / UO → v_x, v_y, w_z → policy → 37 ข้อต่อ

Week 4 – 5

reward = สเปกของงาน

เปลี่ยน weight / เขียน reward ใหม่ แล้ววัดผลด้วย eval

Week 6

หุ่นของเราเอง

URDF ↔ USD, ยืดขา, เทรนใหม่บน Newton

วันนี้: เราจะ (1) ทำให้โจทย์ **ค่อย ๆ ยากขึ้นเอง** ระหว่างเทรน และ (2) เปลี่ยนคนกดปุ่มเป็น **autopilot** ที่ใช้ sensor หาทางออกจากเขาวงกต

THIS WEEK

Curriculum + Maze Challenge

ปีก่อนใช้ Isaac Lab 2 + PhysX + Isaac Sim GUI ปีนี้ port มา Isaac Lab 3 + Newton ทั้งหมด และลองรันทุกคำสั่งแล้ว

CurriculumTermCfg

7.0 · Concept + API

curriculum ทำงานตอนไหน, terrain_levels_vel, built-in ของ Isaac Lab 3

G1-Curriculum-*

7.1 · G1 + 3 terms

reward ramp, push ที่ปรับตามการล้ม และขยายช่วงความเร็ว

G1-Maze-*

7.2 · Maze Challenge

G1 เดินเองในเขาวงกต: สแกนหากำแพง เลือกทาง แล้วสั่งความเร็ว

LECTURE

Curriculum learning

เรียนจากง่ายไปยาก เหมือนเราเรียนเลขก็ไม่ได้เริ่มจากแคลคูลัส

WHY CURRICULUM

ให้หุ่นเจอโจทย์ยากตั้งแต่แรก มันจะล้มจนไม่ได้เรียนอะไร

Sparse reward

reward หายาก

policy ที่ยังสุ่มอยู่แทบไม่เคยทำได้ เลยไม่มีอะไรให้เรียน

Exploration

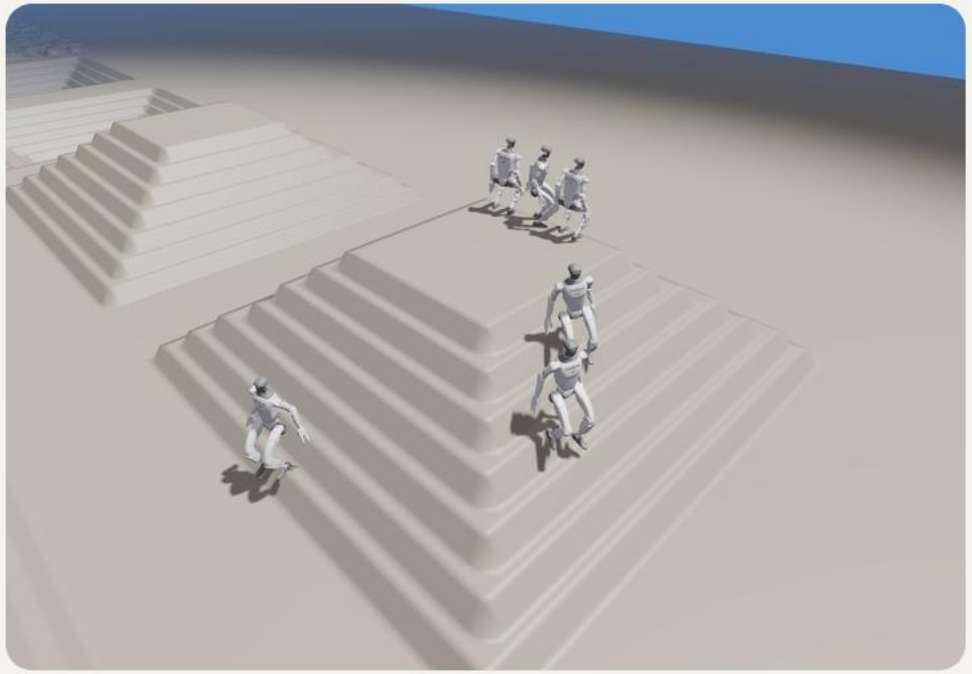
state space ใหญ่

37 ข้อต่อ × พื้นหลายแบบ × ความเร็วหลายค่า สุ่มไปก็ไม่เจอท่าดี

Instability

ล้มตลอด

episode จบใน 1 วินาที ข้อมูลมีแต่ “ท่าล้ม”



บันไดแถวยากสุดของ G1 Rough: ขั้นสูงเกือบ 23 cm ให้หุ่นที่ยังเดินไม่เป็นเริ่มตรงนี้ ล้มแทบทุก episode

Curriculum เริ่มจากโจทย์ที่หุ่น **พอทำได้** แล้วเพิ่มความยาก **เมื่อทำได้ดีพอ** ข้อมูลเลยมีทั้งตอนสำเร็จและตอนพลาด

สรุปในหน้าเดียว

curriculum คือการแก้ค่า config ระหว่างเทรน

1 · ปรับอะไร

เลือก 1 ค่า

weight ของ reward, ช่วงคำสั่งความเร็ว, แรงผลัก, noise หรือแถวของพื้น

2 · ขยับเมื่อไร

ตามเวลา หรือตามผล

ถึง iteration ที่กำหนด หรือเมื่อหุ่นล้มน้อยและตามคำสั่งได้ดี

3 · ขยับแค่ไหน

start → end

กำหนดค่าเริ่ม ค่าสุดท้าย และความเร็วในการเปลี่ยนค่า

หุ่น reset → term ทำงาน → แก้ค่าใน manager → simulation ใช้ค่าใหม่ → `Curriculum/<ชื่อ>`

ค่าที่ปรับได้ · G1 ROUGH

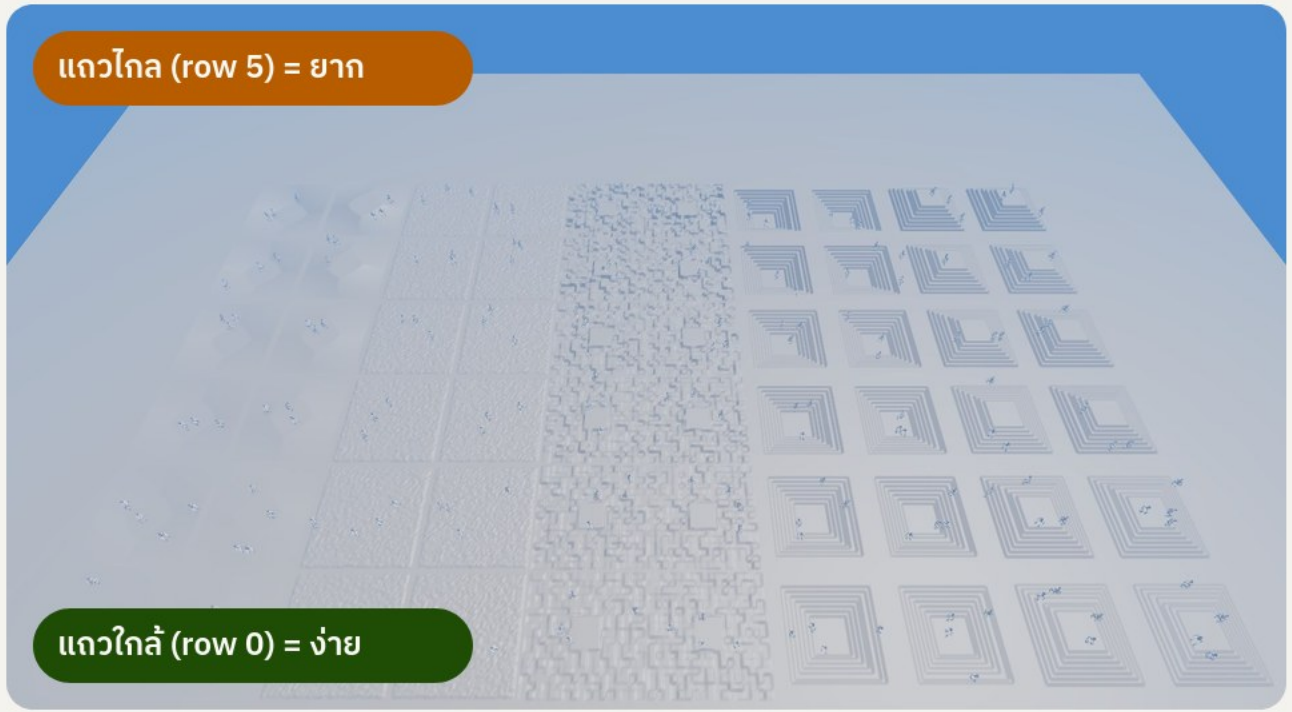
อยากให้ยากขึ้นเรื่องไหน ให้แก้ address นี้

เรื่อง	address	ค่าเดิม	มีผลตอน
ก้าวให้ยาวขึ้น	rewards.feet_air_time.weight	0.25	step ถัดไป
ลดการกระตุก	rewards.action_rate_l2.weight	−0.005	step ถัดไป
เดินเร็วขึ้น	commands.base_velocity.ranges.lin_vel_x	(0, 1.0)	สุ่มคำสั่งรอบถัดไป
เลี้ยวเร็วขึ้น	commands.base_velocity.ranges.ang_vel_z	(−1, 1)	สุ่มคำสั่งรอบถัดไป
ทนแรงผลัก	events.push_robot.params.velocity_range.x	±0.5	ผลักครั้งถัดไป
ทนแรงดันต่อเนื่อง	events.base_external_force_torque.params.force_range	(0, 0)	reset ถัดไป
เริ่มจากท่าเสียสมดุล	events.reset_base.params.velocity_range.roll	±0.5	reset ถัดไป
sensor มีสัญญาณรบกวน	observations.policy.joint_vel.noise.n_max	1.5	step ถัดไป

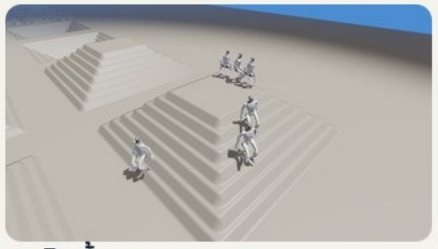
แก้แล้วไม่มีผล: physics_material กับ add_base_mass เป็น event แบบ startup ทำงานครั้งเดียวตอนเริ่มเทรน

TERRAIN GENERATOR · G1-CURRICULUM-*

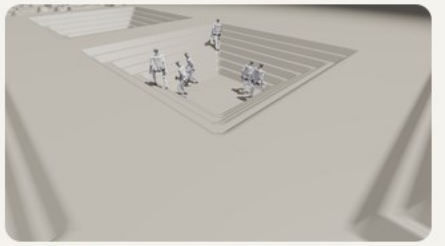
พื้นฝึกคือตาราง: แถว = ความยาก, คอลัมน์ = ชนิดพื้น



แผ่นละ 8 × 8 m · 6 แถว × 10 คอลัมน์ (G1-Curriculum-* ลดจาก 10 × 20 ของ G1 Rough เพื่อให้เทรนบนการ์ดจอ 6 GB ได้)



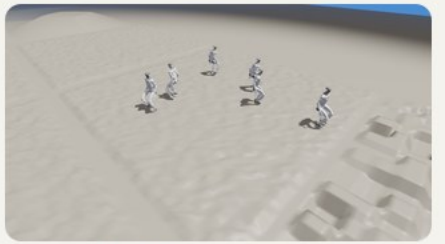
บันไดขึ้น 5 → 23 cm



บันไดลง 5 → 23 cm



กล่อง 5 → 20 cm



ขรุขระ 2 → 10 cm



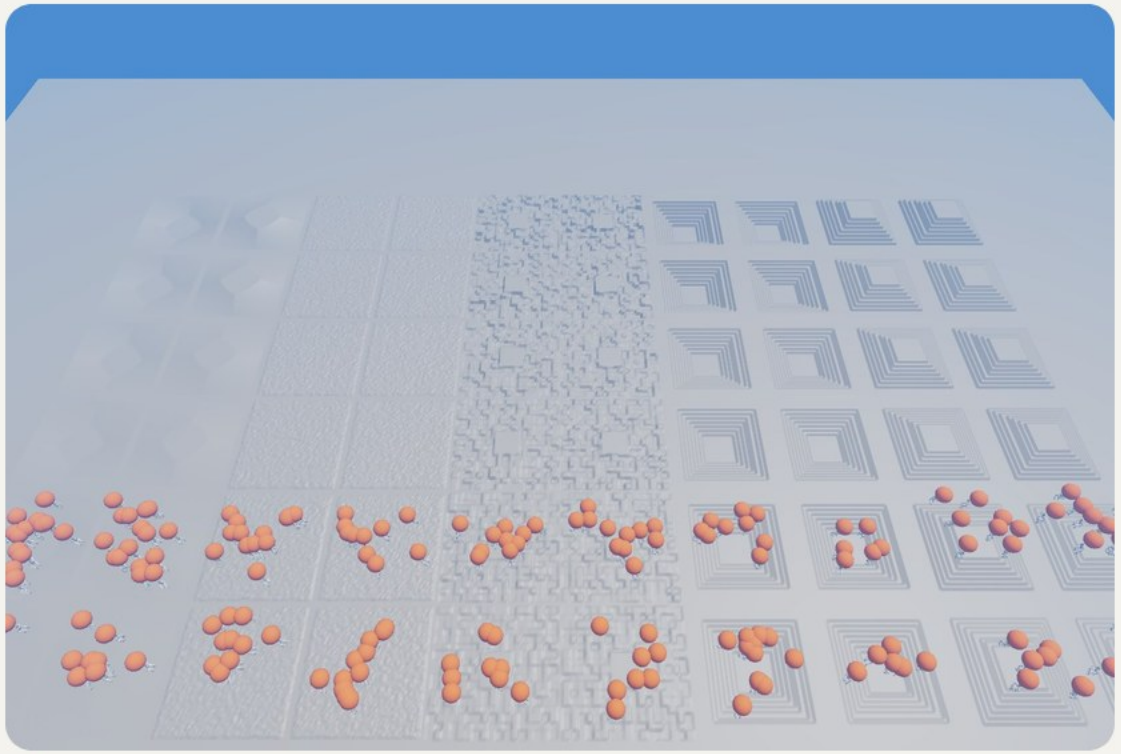
เนินลาด ชัน 0 → 0.4



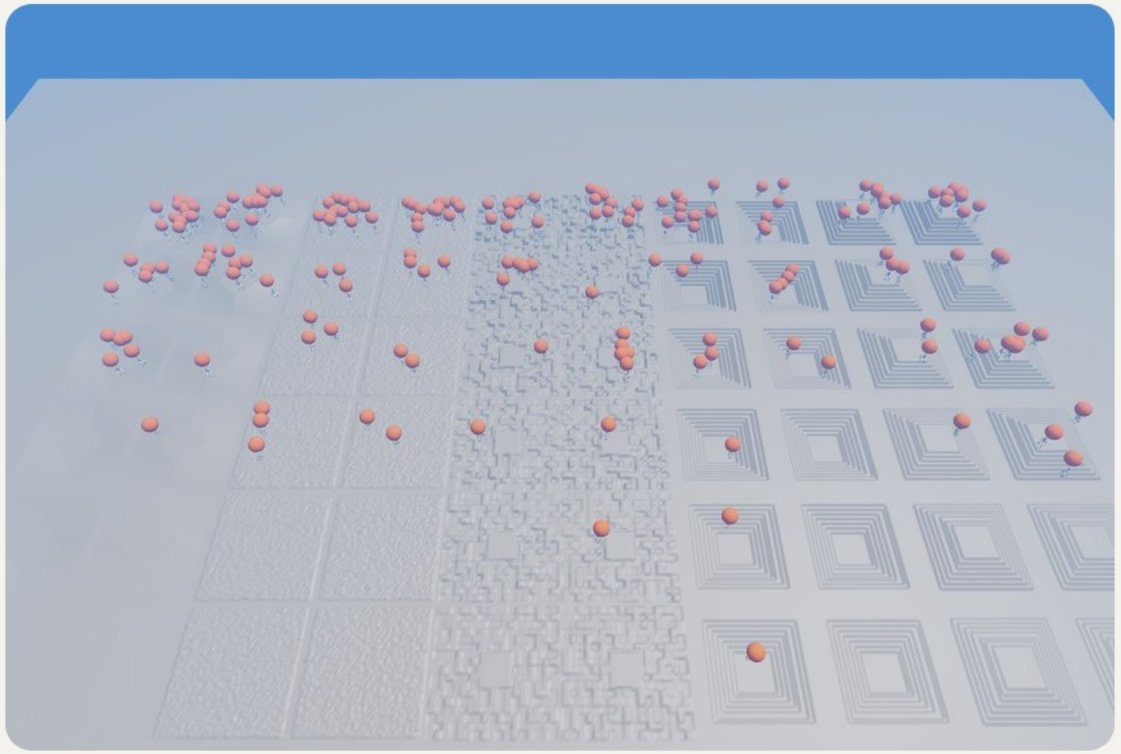
แอ่งลาด ชัน 0 → 0.4

TERRAIN_LEVELS_VEL · ภาพประกอบ

หุ่นที่เดินได้ไกล ค่อย ๆ ย้ายไปแถวที่ยากขึ้น



เริ่มเทรน: ทุกตัวเริ่มแถวง่าย (ใกล้กล้อง)



เทรนไปสักพัก: ตัวที่ผ่าน move_up ไปแถวไกล ตัวที่ไปไม่ถึงครึ่งถอยกลับ

กฎ: เดินเกินครึ่งแผ่น (4 m) ขึ้นแถว เดินไม่ถึงครึ่งของระยะที่สั่ง ลงแถว · กราฟ Curriculum/terrain_levels คือแถวเฉลี่ย

ISAAC LAB 3

term ทำงานยังไง

โค้ดแต่ละส่วนเชื่อมกันอย่างไร และ term อ่านข้อมูลอะไรได้บ้าง

โค้ดเชื่อมกันยังไง

จาก config ไปถึง term ผ่าน 3 จุด

```
# 1. env_cfg.py: เราลงทะเบียน term
self.curriculum.feet_air_time_ramp = CurrTerm(
    func=mdp.RewardWeightRampCurriculum,
    params={"term_name": "feet_air_time", "start_weight": 0.25, ...},
)
```

feet_air_time_ramp คือชื่อกราฟ
func คือโค้ดที่จะเรียก
params คือค่าที่จะส่งให้โค้ดนั้น

```
# 2. Isaac Lab: curriculum_manager.py เรียกทุกครั้งที่มีหุ่น reset
state = term_cfg.func(self._env, env_ids, **term_cfg.params)
```

****params** แตก dict เป็น argument ตามชื่อ ค่าที่ return เก็บใน **state** เพื่อ log

```
# 3. mdp/curriculums.py: โค้ดของเรารับค่า
def __call__(self, env, env_ids, term_name, start_weight, end_weight,
start_step, end_step):
```

ชื่อ argument ต้องตรงกับ key ใน params ทุกตัว

env

ทั้ง environment เข้าถึง reward, command, event และหุ่นได้

env_ids

เลขของหุ่นที่เพิ่งจบ episode เช่น [3, 17, 902]

class หรือ function

class สร้าง __init__ ครั้งเดียว แล้วเรียก __call__ ซ้ำ ส่วน function ถูกเรียกตรง ๆ

ตารางอ้างอิง

ข้อมูลที่ term อ่านจาก env ได้

อยากรู้หรืออยากแก้	เขียน
เทรนมากี่ step แล้ว (÷ 24 = iteration)	`env.common_step_counter`
หุ่นที่เพิ่งจบตัวไหนล้ม	`env.termination_manager.get_term("base_contact")[env_ids]`
หุ่นตัวไหนอยู่รอดจนหมดเวลา	`env.termination_manager.time_outs`
คะแนนสะสมของ episode ที่เพิ่งจบ	`env.reward_manager._episode_sums["track_lin_vel_xy_exp"][env_ids]`
อ่าน หรือแก้ weight ของ reward	`env.reward_manager.get_term_cfg(name)` · `set_term_cfg(name, cfg)`
อ่าน หรือแก้ค่าของ event	`env.event_manager.get_term_cfg("push_robot").params`
อ่าน หรือแก้ช่วงคำสั่งความเร็ว	`env.command_manager.get_term("base_velocity").cfg.ranges`
ตำแหน่งหุ่น	`env.scene["robot"].data.root_pos_w.torch[env_ids, :2]`
แถวพื้นของหุ่นทุกตัว	`env.scene.terrain.terrain_levels`

LAB 7.1

G1 + curriculum 3 แบบ

อ่านโค้ด term 3 ตัว ลองปรับค่า แล้ว fine-tune G1 Rough รอบละประมาณ 5 นาทีบนโน้ตบุ๊ก

ก่อนเริ่ม LAB · โหลดโปรเจกต์

เอา g1_maze_lab ลงเครื่อง แล้วเช็คว่าใช้ได้

```
# 1. ยังไม่เคยโหลด: clone ไว้ที่ ~/Downloads/Simulation_for_Humanoid
$ git clone https://github.com/pannatron/simulation-for-humanoid.git ~/Downloads/Simulation_for_Humanoid
# เคยโหลดแล้ว: อัปเดตเป็นของล่าสุด
$ cd ~/Downloads/Simulation_for_Humanoid && git pull

# 2. ติดตั้ง แล้วเช็ค
$ cd ~/Downloads/Simulation_for_Humanoid/Week07_Curriculum_Maze/g1_maze_lab
$ uv sync                                  # ครั้งแรกครั้งเดียว สร้าง .venv
$ uv run pytest tests -q                   # ต้องได้ 44 passed
$ uv run isaaclab list_envs | grep G1-     # ต้องเห็น 18 tasks
```

Isaac Lab อยู่ที่อื่น?

โปรเจกต์หา Isaac Lab ที่ ~/IsaacLab ถ้าไม่ใช่ ก่อน uv sync ให้รันคำสั่ง sed ในหัวข้อ First-time setup ของ README.md หน้าแรก repo

git pull ไม่ผ่าน?

ใครโหลดไว้ก่อน 30 ต.ค. ประวัติใน repo เปลี่ยน ให้ย้ายโฟลเดอร์เก่าออก แล้ว git clone ใหม่

ในโฟลเดอร์มีอะไร

โค้ดทั้ง 2 lab, policy ที่เทรนแล้ว (pretrained/), สไลด์ PDF ทั้ง Week 7 และ Week พิเศษ

```
g1_maze_lab/
├── pyproject.toml          # entry point isaaclab.tasks
├── pretrained/g1_rough/    # checkpoint G1 Rough (Isaac Lab, Newton)
├── scripts/
│   ├── eval_g1.py          # วัดความเร็ว / ล้ม / แรงผลัก
│   └── maze_autopilot.py   # G1 เดินเองในเขาวงกต
└── src/g1_maze_lab/
    ├── autopilot.py        # กำแพง → ทาง → คำสั่ง
    ├── terrains/maze.py     # sub-terrain เขาวงกต
    └── tasks/
        ├── curriculum/
        │   ├── mdp/curriculums.py
        │   ├── env_cfg.py
        │   └── agents/rsl_rl_ppo_cfg.py
        └── maze/  env_cfg.py · mdp/terminations.py
```

WEEK07 · G1_MAZE_LAB

project เดียว 2 lab

Lab 7.1 · curriculum

9 tasks G1-Curriculum-* (Base, 3 term, Full, ActionRate ของ Ex 7.2 และตัวอย่างเพิ่ม 3 ตัว)

Lab 7.2 · maze

terrain + sensor + autopilot, tasks G1-Maze-* (+ G1-Maze-Hard, Hard9 ของ Challenge 7.5)

```
$ cd Week07_Curriculum_Maze/g1_maze_lab
$ uv sync   # ครั้งแรก
$ uv run isaaclab list_envs | grep G1-
```

ขั้น 0 / 4 · TASKS/CURRICULUM/ENV_CFG.PY

เริ่มจาก Base: ลดขนาด G1 Rough ให้เทรนบนโน้ตบุ๊กได้

0 Base → 1 + Reward ramp → 2 + Push → 3 + Velocity → 4 Full

```python
from ...config.g1.rough_env_cfg import G1RoughEnvCfg

STEPS_PER_IT = 24  # num_steps_per_env ของ PPO: 1 iteration = 24 step

@configclass
class G1CurriculumBaseEnvCfg(G1RoughEnvCfg):
    def __post_init__(self):
        super().__post_init__()  # reward, push, command ของ G1 Rough ครบ
        self.scene.num_envs = 1024
        self.scene.terrain.terrain_generator.num_rows = 6
        self.scene.terrain.terrain_generator.num_cols = 10
        self.scene.terrain.max_init_terrain_level = 3
```

มีอะไรอยู่แล้ว

reward 15 term, ผลัก ±0.5 m/s ทุก 10–15 s, คำสั่ง v_x 0 → 1.0 m/s และ curriculum **terrain_levels** ตัวเดียว

ใช้ทำอะไร

เป็นตัวเทียบ: ทุก run fine-tune 300 it. เท่ากัน ถ้าผลต่างจาก Base แสดงว่าน่าจะมาจาก term ที่เพิ่ม ไม่ใช่จากการเทรนต่อ

```
$ uv run isaaclab train --rl_library rsl_rl --task G1-Curriculum-Base --num_envs 1024 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=300 agent.run_name=base300
TensorBoard: มีแค่ Curriculum/terrain_levels
```

ขั้น 1 / 4 · อ่านโค้ด · REWARDWEIGHTRAMP · ตอนเริ่มเทรน

เตรียม config ของ reward ไว้ครั้งเดียว

```
class RewardWeightRampCurriculum(ManagerTermBase):
```

RewardWeightRampCurriculum ชื่อ term ที่จะเรียกใช้ใน env_cfg.py
ManagerTermBase class แม่ของ Isaac Lab สำหรับ term ที่ต้องเก็บข้อมูลไว้ใน self

```
    def __init__(self, cfg, env):
```

__init__ ทำงานครั้งเดียวตอนสร้าง environment
cfg ค่าที่ตั้งใน CurrTerm(...) รวมถึง params
env environment ทั้งชุด ใช้เข้าถึง reward, event, command และหุ่น

```
        super().__init__(cfg, env)
```

super().__init__ ให้ class แม่เก็บ cfg กับ env ไว้ก่อน

```
        self._term_cfg = env.reward_manager.get_term_cfg(cfg.params["term_name"])
```

cfg.params["term_name"] อ่านชื่อ reward จาก params ได้ "feet_air_time"
env.reward_manager ตัวจัดการ reward ทั้งหมด
get_term_cfg(...) ขอ config ของ reward ตัวนั้น ซึ่งมี weight อยู่ข้างใน
self._term_cfg เก็บไว้ใช้ตอนที่ term ถูกเรียกครั้งต่อไป

ขั้น 1 / 4 · อ่านโค้ด · ทุกครั้งที่มีหุ่น RESET · ตัวอย่าง IT. 100

คำนวณว่าผ่านช่วงที่ตั้งไว้มากี่ส่วนแล้ว

```
def __call__(self, env, env_ids, term_name, start_weight, end_weight, start_step, end_step):
```

`__call__` ถูกเรียกทุกครั้งที่มีหุ่น reset
`env_ids` เลขของหุ่นที่เพิ่งจบ episode แต่ term นี้ไม่ได้ใช้
`term_name ... end_step` มาจาก params: "feet_air_time", 0.25, 1.0, 1200, 3600

```
    frac = (env.common_step_counter - start_step) / max(end_step - start_step, 1)
```

`env.common_step_counter` เทรนมาแล้วกี่ step: 100 × 24 = 2400
`counter - start_step` ผ่านจุดเริ่มมาแล้ว 2400 − 1200 = 1200 step
`end_step - start_step` ช่วงทั้งหมดยาว 3600 − 1200 = 2400 step
`max(..., 1)` กันตัวหารเป็น 0 กรณีตั้งจุดเริ่มกับจุดจบเท่ากัน
`frac` ผลลัพธ์ 1200 ÷ 2400 = **0.5** คือผ่านมาครึ่งทาง

```
    frac = min(max(frac, 0.0), 1.0)
```

`max(frac, 0.0)` ก่อนถึงจุดเริ่ม ค่าจะติดลบ จึงปรับเป็น 0
`min(..., 1.0)` หลังเลยจุดจบ ค่าจะเกิน 1 จึงปรับเป็น 1

ขั้น 1 / 4 · อ่านโค้ด · ตัวอย่าง IT. 100

แปลงสัดส่วนเป็น weight แล้วบันทึกกลับ

```
self._term_cfg.weight = start_weight + frac * (end_weight - start_weight)
```

`end_weight - start_weight` weight ต้องเพิ่มทั้งหมด 1.0 − 0.25 = 0.75
`frac * (...)` เพิ่มมาแล้วครึ่งหนึ่ง 0.5 × 0.75 = 0.375
`start_weight + ...` ได้ weight ปัจจุบัน 0.25 + 0.375 = **0.625**

```
env.reward_manager.set_term_cfg(term_name, self._term_cfg)
```

`set_term_cfg` ส่ง config กลับให้ reward manager step ถัดไปจะใช้ weight ใหม่

```
return self._term_cfg.weight
```

`return` ส่งค่า 0.625 ไปแสดงในกราฟ Curriculum/feet_air_time_ramp

ทำไมถูกเรียกซ้ำได้: ค่าคำนวณจาก counter ทุกครั้ง ไม่ได้บวกเพิ่มจากค่าเดิม เรียกกี่ครั้งใน step เดียวกันก็ได้ผลเท่ากัน

ขั้น 1 / 4 · ปรับค่า · G1-CURRICULUM-REWARD

RewardWeightRamp: ปรับค่าไหน ได้อะไร

```
# env_cfg.py: reward_ramp_term() ปรับตรงนี้
"term_name": "feet_air_time",
"start_weight": 0.25,
"end_weight": 1.0,
"start_step": 50 * STEPS_PER_IT,
"end_step": 150 * STEPS_PER_IT,
```

ปรับ	แล้วได้อะไร
end_weight 1.0 → 2.0	air/ก้าว 0.122 → 0.132 s แต่ v_x ที่สั่ง 2.0 ลด 1.829 → 1.770 m/s
start_weight	ใช้ค่าเดิมของ G1 Rough เพื่อไม่ให้ reward กระโดด
start_step, end_step	กำหนดช่วงที่ค่าเปลี่ยน ถ้าตั้ง 50 กับ 150 กราฟจะขึ้นช่วง it. 5050–5150
term_name	สลับเป็น reward ตัวอื่นได้ทันที

ไม่ต้องแก้โค้ด: ต่อท้ายคำสั่ง train ด้วย

`env.curriculum.feet_air_time_ramp.params.end_weight=2.0`

```
$ uv run isaaclab train --rl_library rsl_rl --task G1-Curriculum-Reward --num_envs 1024 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=300 agent.run_name=reward300
```

เตรียม event ที่จะแก้ และตัวนับที่ต้องจำไว้

```
self._push_cfg = env.event_manager.get_term_cfg(cfg.params.get("event_name", "push_robot"))
```

`cfg.params.get("event_name", ...)` อ่านชื่อ event จาก params ถ้าไม่ได้ตั้งไว้จะใช้ "push_robot"
`env.event_manager` ตัวจัดการ event เช่น การผลักและการ reset
`get_term_cfg(...)` ขอ config ของ event ซึ่งมีช่วงแรงผลักอยู่ใน params

```
self.level = cfg.params.get("start_vel", 0.0)
```

`self.level` แรงผลักปัจจุบัน หน่วย m/s
`"start_vel"` ค่าเริ่มต้นจาก params ใน env_cfg.py ตั้งไว้ 0.0

```
self._ended = 0
self._fell = 0
self._last_update = 0
```

`_ended, _fell` จำนวนหุ่นที่จบ episode และจำนวนที่ล้ม นับสะสมข้ามหลายครั้งที่ถูกเรียก
`_last_update` step ที่ตัดสินใจครั้งล่าสุด
เหตุผล ต้องจำค่าเหล่านี้ไว้ term นี้จึงต้องเป็น class

```
self._apply(cfg.params.get("event_name", "push_robot"), env)
```

`self._apply(...)` เขียนแรงผลักเริ่มต้นลง event ทันที

ขั้น 2 / 4 · อ่านโค้ด · ADAPTIVEPUSH · เมธอด _APPLY

แปลงค่า level เป็นช่วงแรงผลักของ event

```
def _apply(self, event_name, env):
```

`_apply` เมธอดที่เราเขียนเอง เพื่อใช้ซ้ำทั้งตอนเริ่มเทรนและตอนปรับค่า

```
    v = self.level
```

`v` ตั้งชื่อสั้น ๆ ให้แรงผลักปัจจุบัน เช่น 0.5

```
    self._push_cfg.params["velocity_range"] = {"x": (-v, v), "y": (-v, v)}
```

`params["velocity_range"]` ช่วงความเร็วที่ event จะสุ่มมาใช้ผลักหุ่น
`"x", "y"` ผลักได้ทั้งแกนหน้า-หลังและซ้าย-ขวา
`(-v, v)` ถ้า v = 0.5 จะสุ่มระหว่าง −0.5 ถึง 0.5 m/s

```
    env.event_manager.set_term_cfg(event_name, self._push_cfg)
```

`set_term_cfg` ส่ง config กลับให้ event manager การผลักครั้งถัดไปจะใช้ช่วงนี้

สังเกต: term ไม่ได้ผลักหุ่นเอง แค่เปลี่ยนช่วงแรงของ event push_robot ซึ่งผลักหุ่นทุก 10–15 s

ขั้น 2 / 4 · อ่านโค้ด · ทุกครั้งที่มีหุ่น RESET

ขั้นแรก: นับว่าหุ่นจบ episode กี่ตัว และล้มกี่ตัว

```
if env.common_step_counter == 0:
    return self.level
```

`counter == 0` reset ครั้งแรกตอนเปิดโปรแกรม ยังไม่มี episode จบจริง จึงข้ามไป

```
fell = env.termination_manager.get_term(fall_term)[env_ids]
```

`env.termination_manager` ตัวจัดการเงื่อนไขที่ทำให้ episode จบ
`get_term(fall_term)` fall_term คือ "base_contact" ได้ค่าจริง/เท็จของหุ่นทุกตัว
`[env_ids]` เลือกเฉพาะหุ่นที่เพิ่งจบ เช่น [False, True, False]

```
self._ended += len(env_ids)
```

`len(env_ids)` จำนวนหุ่นที่เพิ่งจบ เช่น 3 ตัว

```
self._fell += int(fell.sum())
```

`fell.sum()` นับ True เป็น 1 ตัวอย่างนี้ได้ 1 ตัว
`int(...)` แปลง tensor เป็นเลขธรรมดา

ขั้น 2 / 4 · อ่านโค้ด · สมมติ ENDED 500 ตัว ล้ม 15 ตัว

ขั้นที่สอง: ครบ 20 iteration แล้วตัดสินใจ

```
if env.common_step_counter - self._last_update ⩾ update_every and self._ended > 0:
```

`counter - _last_update` ผ่านมากี่ step แล้วนับจากการตัดสินใจครั้งก่อน
`update_every` 480 step หรือ 20 iteration
`self._ended > 0` ต้องมีหุ่นจบแล้ว เพื่อกันการหารด้วย 0

```
    fall_rate = self._fell / self._ended
```

`fall_rate` 15 ÷ 500 = **0.03** คือล้ม 3 %

```
    if fall_rate < raise_below:
        self.level = min(self.level + step, max_vel)
```

`raise_below` 0.05 ดังนั้น 0.03 เข้าเงื่อนไขนี้
`self.level + step` เพิ่มแรงผลัก 0.1 m/s
`min(..., max_vel)` ไม่ให้เกินเพดาน 1.5 m/s

```
    elif fall_rate > lower_above:
        self.level = max(self.level - step, 0.0)
```

`lower_above` 0.2 ถ้าล้มเกิน 20 % จะลดแรงผลัก 0.1 m/s
`max(..., 0.0)` ไม่ให้แรงผลักติดลบ ถ้าล้ม 5–20 % จะคงค่าเดิม

ขั้น 2 / 4 · อ่านโค้ด · จบการตัดสินใจ

ขั้นสุดท้าย: เขียนค่าใหม่ แล้วเริ่มนับรอบใหม่

```
    self._apply(event_name, env)
```

`self._apply` เขียน level ใหม่ลง event ถ้า level เป็น 0.6 ช่วงจะเป็น ±0.6 m/s

```
    self._ended, self._fell, self._last_update = 0, 0, env.common_step_counter
```

`0, 0` ล้างตัวนับ เพื่อให้รอบหน้าวัดจากข้อมูลใหม่เท่านั้น
`env.common_step_counter` จำ step นี้ไว้ แล้วนับต่อไปอีก 20 iteration

```
return self.level
```

`return` อยู่นอก if จึงทำงานทุกครั้ง ค่าแสดงในกราฟ Curriculum/push_strength

ทำไมต้องมี update_every: term ถูกเรียกหลายครั้งต่อ iteration ถ้าเพิ่มค่าทุกครั้ง แรงผลักจะพุ่งถึงเพดานในไม่กี่ iteration

ขั้น 2 / 4 · ปรับค่า · G1-CURRICULUM-PUSH

AdaptivePush: ปรับค่าไหน ได้อะไร

```
# env_cfg.py: push_term() ปรับตรงนี้
"start_vel": 0.0, "max_vel": 1.5, "step": 0.1,
"raise_below": 0.05, "lower_above": 0.2,
"update_every": 20 * STEPS_PER_IT,
```

ปรับ	แล้วได้อะไร
start_vel 0 → 0.5	**ล้มลดจาก 13–17 เหลือ 6–7 ครั้ง** ใกล้ Base (5–9)
max_vel	เพดานแรงผลัก เริ่ม 0.5 จะถึง 1.5 ใน 200 it.
step, update_every	เปลี่ยนค่าเร็วแค่ไหน ตอนนี้เพิ่ม 0.1 ทุก 20 it.
raise_below, lower_above	ล้มต่ำกว่า 5 % เพิ่ม เกิน 20 % ลด ระหว่างนั้นคงเดิม

ลองเลย: ต่อท้ายคำสั่ง train ด้วย

`env.curriculum.push_strength.params.start_vel=0.5`

```
$ uv run isaaclab train --rl_library rsl_rl --task G1-Curriculum-Push --num_envs 1024 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=300 agent.run_name=push300
```

ขั้น 3 / 4 · อ่านโค้ด · COMMANDRANGE · ตอนเริ่มเทรน

เก็บช่วงคำสั่งความเร็ว แล้วตั้งค่าเพดานเริ่มต้น

```
self._ranges = env.command_manager.get_term(cfg.params.get("command_name", "base_velocity")).cfg.ranges
```

env.command_manager ตัวจัดการคำสั่ง เช่น ความเร็วที่สั่งให้หุ่นเดิน
get_term("base_velocity") ขอคำสั่งความเร็ว
.cfg.ranges ช่วงที่ใช้สุ่มคำสั่ง เช่น lin_vel_x และ ang_vel_z

```
self._ranges.lin_vel_x = (self._ranges.lin_vel_x[0], cfg.params.get("start_max", 0.5))
```

lin_vel_x[0] คงค่าต่ำสุดเดิมไว้ที่ 0
"start_max" ค่าเพดานเริ่มต้น ใน env_cfg.py ตั้งไว้ 1.0
(..., ...) แก้ค่าใน tuple ทีละตัวไม่ได้ จึงสร้างใหม่ ผลลัพธ์คือ (0, 1.0)

```
self._sum, self._count, self._last_update = 0.0, 0, 0
```

_sum, _count คะแนนรวม และจำนวนหุ่นที่นับแล้ว
_last_update step ที่ตัดสินใจครั้งล่าสุด

ขั้น 3 / 4 · อ่านโค้ด · ทุกครั้งที่มีหุ่น RESET

ขั้นแรก: สะสมคะแนนการเดินของหุ่นที่เพิ่งจบ

```
if env.common_step_counter == 0 or len(env_ids) == 0:
    return self._ranges.lin_vel_x[1]
```

เงื่อนไข reset ครั้งแรก หรือไม่มีหุ่นจบ จึงยังไม่มีอะไรให้นับ
lin_vel_x[1] เพดานความเร็วปัจจุบัน

```
sums = env.reward_manager._episode_sums[reward_term][env_ids]
```

_episode_sums คะแนนสะสมของ episode ปัจจุบัน แยกตาม reward
[reward_term] "track_lin_vel_xy_exp" คือคะแนนเดินตามความเร็วที่สั่ง
[env_ids] เฉพาะหุ่นที่เพิ่งจบ อ่านได้เพราะ curriculum ทำงานก่อนล้างคะแนน

```
self._sum += float(sums.sum()) / env.max_episode_length_s
```

sums.sum() รวมคะแนนของหุ่นทุกตัวที่เพิ่งจบ
/ max_episode_length_s หาร 20 s ให้เป็นคะแนนต่อวินาที ค่าเต็มของแต่ละตัวจึงเท่ากับ weight

```
self._count += len(env_ids)
```

_count นับจำนวนหุ่น เพื่อหาค่าเฉลี่ยตอนตัดสินใจ

ขั้น 3 / 4 · อ่านโค้ด · สมมติคะแนนเฉลี่ย 0.85

ขั้นที่สอง: เดินตามคำสั่งได้ดีพอ จึงขยายช่วง

```
if env.common_step_counter - self._last_update ⩾ update_every:
```

`update_every` ตัดสินใจทุก 480 step หรือ 20 iteration

```
    weight = env.reward_manager.get_term_cfg(reward_term).weight
```

`.weight` weight ของ track_lin_vel_xy_exp คือ 1.0 ซึ่งเป็นคะแนนเต็มต่อวินาที

```
    if self._sum / self._count > threshold * weight:
```

`_sum / _count` คะแนนเฉลี่ยต่อตัว ตัวอย่างนี้ 0.85
`threshold * weight` เกณฑ์ผ่าน 0.8 × 1.0 = 0.8 ดังนั้นผ่าน

```
        lo, hi = self._ranges.lin_vel_x
        self._ranges.lin_vel_x = (lo, min(hi + step, final_max))
```

`lo, hi` แยกช่วงเดิมออกเป็น 0 และ 1.0
`hi + step` เพิ่มเพดานเป็น 1.1 m/s
`min(..., final_max)` ไม่ให้เกิน 2.0 m/s มีผลเมื่อสุ่มคำสั่งรอบถัดไป

```
    self._sum, self._count, self._last_update = 0.0, 0, env.common_step_counter
return self._ranges.lin_vel_x[1]
```

`0.0, 0` ล้างตัวนับเพื่อเริ่มรอบใหม่
`return` เพดานปัจจุบันแสดงในกราฟ Curriculum/command_vel_x

ขั้น 3 / 4 · ปรับค่า · G1-CURRICULUM-VELOCITY

CommandRange: ปรับค่าไหน ได้อะไร

```
# env_cfg.py: velocity_term() ปรับตรงนี้
"start_max": 1.0, "final_max": 2.0, "step": 0.1,
"threshold": 0.8,
"update_every": 20 * STEPS_PER_IT,
```

ปรับ	แล้วได้อะไร
threshold 0.8	ความเร็วต้องคลาดเคลื่อนไม่เกินราว 0.24 m/s ถ้าลดค่านี้ ช่วงจะขยายง่ายขึ้น
final_max	เพดานคำสั่ง v_x ตั้ง 2.0 แล้ว run นี้ถึง 2.0 ที่ it. 5240
step, update_every	ความชันของกราฟ ตอนนี้ขึ้นได้สูงสุด 0.1 ทุก 20 it.

อย่าเอา update_every ออก: term ถูกเรียกหลายครั้งต่อ iteration ฉบับแรกที่ไม่มีค่านี้ ขยายจาก 1.0 เป็น 2.0 ภายใน 4 it.

ถ้าล้มก็ไม่ผ่านเกณฑ์: หุ่นที่ล้มตอน 10 s ได้คะแนนสูงสุดแค่ครึ่งเดียว

```
$ uv run isaaclab train --rl_library rsl_rl --task G1-Curriculum-Velocity --num_envs 1024 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=300 agent.run_name=vel300
```

ขั้น 4 / 4 · G1-CURRICULUM-FULL

Full = Base + 3 term ที่ลองทีละตัวมาแล้ว

```python
@configclass
class G1CurriculumFullEnvCfg(G1CurriculumBaseEnvCfg):
    def __post_init__(self):
        super().__post_init__()
        self.curriculum.feet_air_time_ramp = reward_ramp_term()
        self.curriculum.push_strength = push_term()
        self.curriculum.command_vel_x = velocity_term()

# __init__.py: register ทุก task ด้วย loop เดียว
for _name in ("Base", "Reward", "Push", "Velocity", "Full", ...):
    gym.register(id=f"G1-Curriculum-{_name}", ...
        env_cfg_entry_point=f"...env_cfg:G1Curriculum{_name}EnvCfg")
```

ทำไมลองทีละตัวก่อน: ถ้ารวม 3 term แล้วผลแย่ จะไม่รู้ว่าเกิดจากตัวไหน การรันแยกช่วยให้เห็นสาเหตุ เช่น หุ่นล้มมากขึ้นเมื่อผลัก 2.0 m/s พบใน Push (16 ครั้ง)

task	air ramp	push	v_x range
Base	–	–	–
Reward	✓	–	–
Push	–	✓	–
Velocity	–	–	✓
Full	✓	✓	✓

ทุก task มี terrain_levels จาก G1 Rough อยู่แล้ว

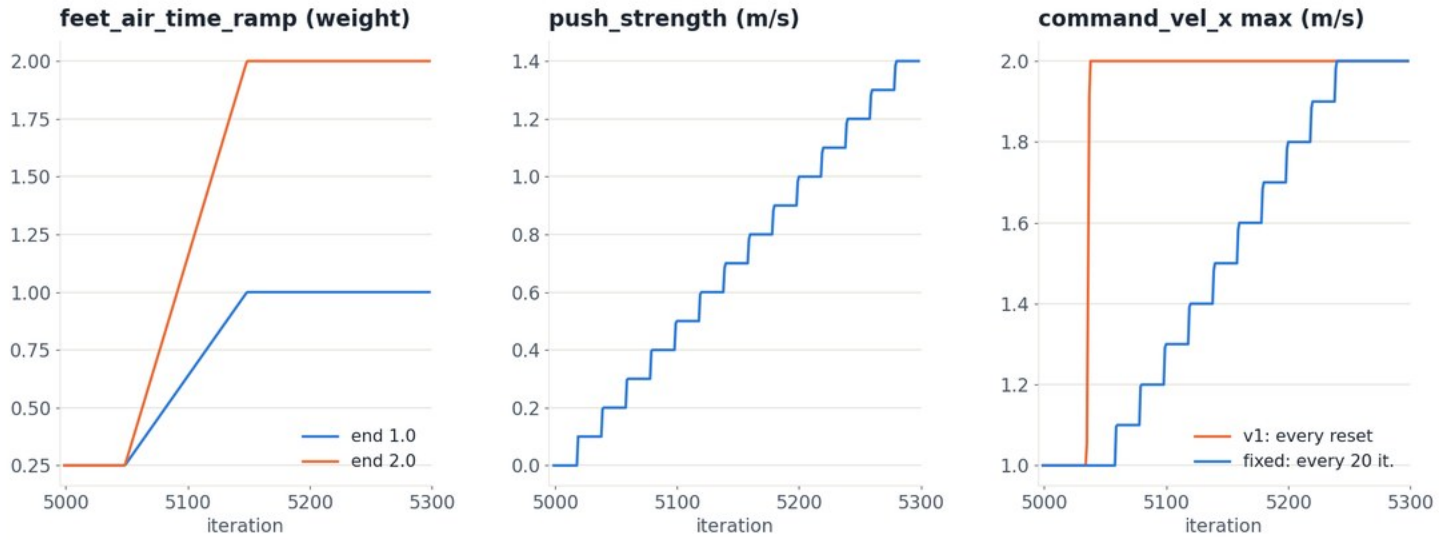
`term` ไม่ชนกัน

3 ตัวแก้คนละ manager จึงเปิดพร้อมกันได้

```
$ uv run isaaclab train --rl_library rsl_rl --task G1-Curriculum-Full --num_envs 1024 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=300 agent.run_name=full300
$ uv run tensorboard --logdir logs/rsl_rl/g1_curriculum
```

TENSORBOARD · CURRICULUM/*

curriculum ทำงานจริงไหม ดูกราฟที่ term return



`feet_air_time_ramp`

0.25 → 1.0

ขึ้นเป็นเส้นตรงช่วง it. 5050 → 5150 ตามที่ตั้งไว้

`push_strength`

0 → 1.4 m/s

ขึ้นทีละ 0.1 ทุก 20 it. ไม่เคยลดเลย (ล้ม < 5 % ตลอด) จบที่ 1.4

`command_vel_x`

1.0 → 2.0 m/s

ขึ้นทีละ 0.1 ทุก 20 it. ถึง 2.0 ที่ it. 5240 (ฉบับแรกถึงใน 4 it.)

Curriculum/* จาก TensorBoard, fine-tune 300 it. (log นับ iteration ต่อจาก checkpoint 5000 → 5298)

ผล LAB 7.1: 300 IT. × 1,024 ตัว, ~5.5 นาที/รัน

curriculum ขึ้นครบตามแผน แต่พฤติกรรมเปลี่ยนไม่มาก

run	ค่าสุดท้ายของ curriculum	Mean reward	v_x จริง สั่ง 1.0	v_x จริง สั่ง 2.0	air/ก้าว (s) สั่ง 0.8	ล้มเมื่อผลัก 1.0 m/s	ล้มเมื่อผลัก 2.0 m/s
pretrained (ไม่ fine-tune)	–	–	0.845	1.867	0.110	1	9
Base (ไม่มี term ใหม่)	air 0.25 · push 0.5* · v_x≤1.0	23.2	0.834	1.846	0.119	1	8
Reward ramp	air 1.0 · push 0.5* · v_x≤1.0	23.8	0.833	1.829	0.122	1	–
Adaptive push	air 0.25 · push 1.4 · v_x≤1.0	22.5	0.802	1.845	0.120	2	16
Command range	air 0.25 · push 0.5* · v_x≤2.0	19.7	0.842	1.794	0.111	0	–
Full (3 terms)	air 1.0 · push 1.4 · v_x≤2.0	19.9	0.846	1.834	0.116	0	19

* push ±0.5 m/s ทุก 10–15 s คือค่าเดิมของ G1 Rough วัดด้วย eval_g1.py บน rough terrain (play mode), 64 ตัว × 9 s ต่อคำสั่ง, ผลักทุก 2 s ทิศสุ่ม รันแค่ครั้งเดียว ต่างกัน 1–2 ครั้งถือว่าเท่ากัน

อ่านผลให้ถูก

Mean reward ของแต่ละ run เทียบกันตรง ๆ ไม่ได้

โจทย์เปลี่ยน

reward เปลี่ยนตาม curriculum

Velocity / Full ได้ Mean reward 19.7–19.9 ต่ำกว่า Base 23.2 เพราะต้องตามคำสั่งถึง 2.0 m/s โจทย์ยากขึ้น ตัว policy ไม่ได้แย่ลง

วัดด้วย `eval`

เทียบที่คำสั่งเดียวกัน

สั่ง 1.0 / 2.0 m/s ทุก run ได้ 0.80–0.85 / 1.77–1.87 m/s เพราะ 300 it. บน policy ที่เทรนมาแล้ว 5,000 it. เปลี่ยนได้น้อย

`curriculum` มีข้อเสีย

ไม่ได้มีแต่ข้อดี

Push ที่เริ่ม 0 m/s ง่ายกว่าค่าเดิม 0.5 และล้ม 13–17 ครั้งเมื่อผลัก 2.0 m/s พอเริ่ม 0.5 **เหลือ 6–7 ครั้ง** ใกล้ Base (5–9)

Lab 7.1 สอนว่า curriculum ทำงานตามโค้ด (กราฟขึ้นตามแผน) แต่ต้องวัดพฤติกรรมด้วยเสมอ จุดเริ่มต้องไม่ง่ายกว่าที่ policy เคยเจอ และถ้า curriculum ไม่เคย “ถอย” เลย แปลว่าเกณฑ์หลวมไป

ลงมือเอง

เพิ่ม curriculum ของตัวเอง

แก้ไฟล์ไหน พิมพ์อะไร และถ้า error ต้องกลับไปดูตรงไหน

SRC/G1_MAZE_LAB/TASKS/CURRICULUM/

อยากเพิ่มอะไร ต้องแก้ไฟล์ไหน

อยากทำ	mdp/curriculums.py	mdp/__init__.pyi	env_cfg.py	__init__.py
เปลี่ยนตัวเลข ของ term เดิม	–	–	แก้ค่าใน params	–
เปิด term เดิมเพิ่ม ใน task เดิม	–	–	เพิ่ม 1 บรรทัด	–
สร้าง task ใหม่ จาก term ที่มีอยู่	–	–	เพิ่ม class	เติมชื่อ task
เขียน term ใหม่ พร้อม task ใหม่	เขียน term	เติมชื่อ 2 จุด	เพิ่ม class	เติมชื่อ task

```
# __init__.py บรรทัด 12: ชื่อใน tuple กำหนด 2 อย่าง
for _name in ("Base", ..., "Turn"):
    id=f"G1-Curriculum-{_name}"  # ชื่อที่ใส่ใน --task
    f"...env_cfg:G1Curriculum{_name}EnvCfg"  # ชื่อ class
```

ทางลัด: ถ้าวาง term ไว้ใน env_cfg.py เลย ไม่ต้องแก้ 2 ไฟล์ใน mdp/

ทำตามได้เลย · แบบที่ 1 · ใช้ RAMP ที่มีอยู่

เพิ่ม task “เลี้ยวเร็วขึ้น” แก้ 2 จุด

จุดที่ 1

env_cfg.py

วางท้ายไฟล์

```
@configclass
class G1CurriculumTurnEnvCfg(G1CurriculumBaseEnvCfg):
    def __post_init__(self):
        super().__post_init__()
        self.curriculum.turn = ramp("commands.base_velocity.ranges.ang_vel_z",
            0, 20, (-0.5, 0.5), (-1.0, 1.0))
```

จุดที่ 2

__init__.py

บรรทัด 12

```
for _name in ("Base", ..., "Difficulty", "Turn"):  # เติม "Turn"
```

รัน

ใช้เวลาราว 1 นาที

```
$ uv run isaaclab train --rl_library rsl_rl --task G1-Curriculum-Turn --num_envs 64 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=30 agent.run_name=turn
```

ค่าที่ส่งให้ ramp: address, it. เริ่ม, it. จบ, ค่าเริ่ม, ค่าจบ

ถูกต้องเมื่อเห็น `Curriculum/turn` ในหน้าจอ และค่าเพิ่มจาก 0.51 เป็น 1.00

ทำตามได้เลย · แบบที่ 2 · เขียน TERM เอง

term ใหม่: หุ่นอยู่แถวยากขึ้น ยิ่งสั่งเลี้ยวเร็วขึ้น

จุดที่ 1 · mdp/curriculums.py วางท้ายไฟล์

```python
def turn_by_terrain(env, env_ids, max_turn: float):

    level = env.scene.terrain.terrain_levels.float().mean().item()

    turn = 0.5 + (max_turn - 0.5) * min(level / 5, 1.0)

    env.command_manager.get_term("base_velocity").cfg.ranges.ang_vel_z = (-turn, turn)

    return turn
```

ตั้ง argument เองได้

อ่าน: แถวเฉลี่ย เช่น 2.5

คำนวณ: 0.5 + 0.5 × 0.5 = 0.75

เขียน: ช่วงคำสั่งเป็น ±0.75

return: ขึ้นกราฟ

จุดที่ 2 · mdp/__init__.pyi

```python
# ใน __all__ = [...]
"turn_by_terrain",
# ใน from .curriculums import (...)
turn_by_terrain,
```

จุดที่ 3 · env_cfg.py ใน __post_init__

```python
self.curriculum.turn = CurrTerm(
    func=mdp.turn_by_terrain,
    params={"max_turn": 1.0})
```

จุดที่ 4 · __init__.py

เติม "Turn" ใน tuple แล้วรัน คำสั่งเดียวกับหน้าก่อน จะเห็นกราฟราว 0.75

โครงสำหรับ TERM ที่ปรับตามผล

ต้องจำข้อมูลข้ามรอบ ใช้โครง class นี้

```python
class MyTerm(ManagerTermBase):
    def __init__(self, cfg, env):
        super().__init__(cfg, env)
        self.level = cfg.params["start"]
        self._good, self._n, self._last = 0, 0, 0

    def __call__(self, env, env_ids, start, end, step, every):
        if env.common_step_counter == 0:
            return self.level
        # A. สะสม: หุ่นตัวไหนทำได้ดี
        ok = env.termination_manager.time_outs[env_ids]
        self._good += int(ok.sum()); self._n += len(env_ids)
        # B. ตัดสินใจเป็นรอบ
        if env.common_step_counter - self._last ⩾ every:
            if self._good / max(self._n, 1) > 0.9:
                self.level = min(self.level + step, end)
            # C. เขียนค่าใหม่
            env.event_manager.get_term_cfg("push_robot").params[
                "velocity_range"]["x"] = (-self.level, self.level)
            self._good, self._n, self._last = 0, 0, env.common_step_counter
        return self.level
```

A · เกณฑ์วัด

เปลี่ยนได้ เช่น นับคนล้ม, ระยะเดิน หรือคะแนน reward

B · ความถี่

ส่ง every = 20 * 24 เพื่อตัดสินใจทุก 20 it. อย่าเอาเงื่อนไขนี้ออก

C · ค่าที่จะแก้

ดูวิธีเขียนค่าได้จากหน้า “ข้อมูลที่ term อ่านจาก env ได้”

ERROR จริงจากการตั้งใจทำผิด

เห็นข้อความนี้ กลับไปแก้จุดนี้

ข้อความ error	สาเหตุ	แก้ที่
`NameNotFound: Environment `G1-Curriculum-Turn` doesn't exist`	ไม่ได้เติมชื่อ task	__init__.py บรรทัด 12
`env_cfg' has no attribute 'G1CurriculumTurnEnvCfg'`	ไม่มี class หรือชื่อไม่ตรง	ชื่อ class ใน env_cfg.py
`No ...curriculum.mdp attribute turn_by_terrain`	ไม่ได้ประกาศ term	mdp/__init__.pyi ทั้ง 2 จุด
`The term 'turn' expects mandatory parameters: ['max_turn'] ... but received: ['max_turnn']`	key ไม่ตรง argument	params ใน env_cfg.py
`'CommandsCfg' object has no attribute 'base_velocit'`	address พิมพ์ผิด	address ใน ramp(...)
`Reward term 'feet_air_tim' not found.`	ชื่อ reward ผิด	term_name ใน params

ไม่มี error แต่กราฟไม่ขึ้น: term ไม่ได้ return ตัวเลข ส่วนกราฟไม่ขยับให้เช็คว่าตั้ง iteration เกินความยาวรอบหรือเปล่า

เขียนแบบไหนดี

เลือกวิธีที่เขียนโค้ดน้อยที่สุดและยังได้ผลที่ต้องการ

วิธี	เขียนอะไร	ขึ้นกราฟ	ใช้เมื่อ
จูนค่า term เดิม	env.curriculum.<ชื่อ>.params.end_weight=2.0	ตาม term เดิม	แค่ลองตัวเลข
modify_term_cfg	modify_fn 1 ฟังก์ชัน + address	ไม่ขึ้น	ไม่ต้องการกราฟ
ramp(...)	1 บรรทัดต่อ 1 ค่า	ขึ้น	ค่อย ๆ เปลี่ยนค่าตามเวลา
function	def f(env, env_ids, ...) แล้ว return ค่า	ขึ้น	ไม่ต้องจำอะไรข้ามรอบ
class	ManagerTermBase มี __init__ + __call__	ขึ้น	ต้องสะสมข้อมูล เช่น นับคนล้ม

จำง่าย ๆ: กราฟขึ้นเมื่อ term return ค่า ไม่เกี่ยวว่าเขียนเป็น function หรือ class เช่น terrain_levels_vel เป็น function และมีกราฟ

เช็คก่อนเทรน

อาการที่เจอบ่อย และจุดที่ต้องแก้

อาการ	สาเหตุ	แก้
ค่าพุ่งถึงเพดานในไม่กี่ it.	term ถูกเรียกทุกครั้งที่มีหุ่น reset	คำนวณจาก counter หรือใส่ update_every
กราฟไม่ขยับเลย	ตั้ง iteration เกินความยาวรอบ และ counter เริ่ม 0 ทุกรอบ	นับจากรอบนี้ เช่น รอบ 300 it. ตั้งไม่เกิน 300
กราฟขึ้น แต่หุ่นไม่ต่าง	แก้ event แบบ startup	เลือก event แบบ reset หรือ interval
ไม่มีกราฟ Curriculum/ชื่อ	term return None	return ตัวเลข หรือใช้ Ramp
error ตอนใช้ lin_vel_x[1]	address แก้สมาชิกใน tuple ไม่ได้	ส่ง tuple ทั้งก้อน
หุ่นล้มมากกว่าเดิมหลังเทรน	start ง่ายกว่าค่าที่ policy เคยเจอ	ตั้ง start เท่าค่าเดิม
ค่ากลับไปจุดเริ่มเมื่อเทรนต่อ	ระดับ curriculum ไม่ได้เก็บใน checkpoint	ตั้ง start เป็นค่าล่าสุดเมื่อ resume

นอกเหนือจาก 3 TERM

ตัวอย่างเพิ่มเติม

ค่าอื่นบน G1 ที่ทดสอบแล้ว และแนวทางทำการบ้านกับ Challenge 7.5

อ่านโค้ด · MDP/CURRICULUMS.PY · ตัวอย่าง IT. 40

Ramp: คำนวณว่าผ่านช่วงที่ตั้งไว้มากี่ส่วนแล้ว

```
class Ramp(modify_term_cfg):
```

modify_term_cfg term สำเร็จรูปของ Isaac Lab ที่แก้ค่าตาม address ได้
Ramp รับความสามารถนี้มา แล้วเพิ่มการคำนวณค่าและ return ขึ้นกราฟ

```
    def __call__(self, env, env_ids, address, start_it, end_it, start, end):
```

address ตำแหน่งของค่าที่จะแก้ เช่น "rewards.action_rate_l2.weight"
start_it, end_it iteration ที่เริ่มและจบ เช่น 0 กับ 80
start, end ค่าเริ่มและค่าสุดท้าย เช่น −0.005 กับ −0.02

```
        it = env.common_step_counter / STEPS_PER_IT
```

STEPS_PER_IT 1 iteration มี 24 step ดังนั้น 960 ÷ 24 = iteration 40

```
        t = min(max((it - start_it) / (end_it - start_it), 0.0), 1.0)
```

(it - start_it) / (...) ผ่านมาแล้ว (40 − 0) ÷ (80 − 0) = **0.5**
min(max(..., 0.0), 1.0) จำกัดค่าให้อยู่ระหว่าง 0 ถึง 1

อ่านโค้ด · MDP/CURRICULUMS.PY · ตัวอย่าง IT. 40

Ramp: คำนวณค่าใหม่ เขียนลง config แล้วส่งขึ้นกราฟ

```
        def mix(a, b):
            return a + t * (b - a)
```

mix หาค่าระหว่าง a กับ b ตามสัดส่วน t: mix(−0.005, −0.02) = −0.0125

```
        value = tuple(map(mix, start, end)) if isinstance(start, tuple) else mix(start, end)
```

isinstance(start, tuple) เช็คว่าค่าที่ตั้งเป็นช่วง เช่น (−0.5, 0.5) หรือไม่
map(mix, start, end) ถ้าเป็นช่วง เรียก mix ทีละคู่: (−0.5, 0.5) ถึง (−1, 1) ได้ (−0.75, 0.75)
else mix(start, end) ถ้าเป็นเลขเดี่ยว เรียก mix ครั้งเดียว

```
        super().__call__(env, env_ids, address, lambda *_: value)
```

super().__call__ ให้ modify_term_cfg หาตำแหน่งตาม address แล้วเขียนค่า
address rewards.x.weight จะถูกแปลงเป็น reward_manager.cfg.x.weight
lambda *_: value ฟังก์ชันที่รับค่าใดก็ได้ แต่คืน value เสมอ เพราะเราไม่ใช้ค่าเดิม

```
        return value[1] if isinstance(value, tuple) else value
```

value[1] กราฟรับได้เลขเดียว ถ้าเป็นช่วงจึงส่งค่าสูงสุด เช่น 0.75

ตัวอย่าง · --TASK G1-CURRICULUM-KNOBS

1 โจทย์ใช้ 1 บรรทัด และค่าเปลี่ยนตามที่ตั้งจริง

โจทย์	บรรทัดที่เขียน	ค่าที่อ่านจาก manager
ลดการกระตุก	`ramp("rewards.action_rate_l2.weight", 0, 80, -0.005, -0.02)`	−0.005 → −0.0125 → −0.02
เลี้ยวเร็วขึ้น	`ramp("commands.base_velocity.ranges.ang_vel_z", 0, 80, (-.5, .5), (-1., 1.))`	สุ่มได้สูงสุด 0.50 → 1.00
ทนแรงผลัก	`ramp("events.push_robot.params.velocity_range.x", 0, 80, (-.5, .5), (-1., 1.))`	±0.5 → ±1.0 m/s
ทนแรงดันต่อเนื่อง	`ramp("events.base_external_force_torque.params.force_range", 20, 100, (0., 0.), (-20., 20.))`	0 → ±20 N
เริ่มจากท่าเสียสมดุล	`ramp("events.reset_base.params.velocity_range.roll", 0, 80, (-.1, .1), (-.5, .5))`	±0.1 → ±0.5 rad/s
sensor มีสัญญาณรบกวน	`ramp("observations.policy.joint_vel.noise.n_max", 0, 80, 0.5, 1.5)` และ n_min คู่กัน	0.5 → 1.5 rad/s

อ่านผลให้ถูก: รอบนี้เปิดทุกค่าพร้อมกันเพื่อยืนยันว่าโค้ดทำงาน หุ่นล้ม 0.5 % แต่ Mean reward เหลือ −5.8 เพราะ action_rate −0.02 หักคะแนนมาก จึงยังบอกไม่ได้ว่าเดินดีขึ้นหรือแย่ลง

อ่านโค้ด · FUNCTION · --TASK G1-CURRICULUM-TERRAINRULE

terrain_no_fall: หาว่าหุ่นเดินได้ไกลแค่ไหน

```
def terrain_no_fall(env, env_ids):
    terrain = env.scene.terrain
```

`def` เป็น function เพราะอ่านทุกอย่างได้ตอนนั้น ไม่ต้องจำข้ามรอบ
`env.scene.terrain` พื้นทั้งหมด รวมถึงแถวปัจจุบันของหุ่นแต่ละตัว

```
    if env.common_step_counter == 0:
        return terrain.terrain_levels.float().mean()
```

`counter == 0` reset แรกตอนเปิดโปรแกรม จึงยังไม่ย้ายแถว
`.float().mean()` แปลงเลขแถวเป็นทศนิยม แล้วหาค่าเฉลี่ยของหุ่นทุกตัว

```
    pos = env.scene["robot"].data.root_pos_w.torch[env_ids, :2]
```

`root_pos_w` ตำแหน่งลำตัวในพิกัดโลก
`.torch` แปลงเป็น tensor ของ PyTorch
`[env_ids, :2]` เฉพาะหุ่นที่เพิ่งจบ และเอาแค่แกน x, y

```
    walked = torch.linalg.norm(pos - env.scene.env_origins[env_ids, :2], dim=1)
```

`env_origins` จุดเริ่มของหุ่นแต่ละตัว
`norm(..., dim=1)` ระยะห่างจากจุดเริ่มถึงจุดจบ แยกทีละตัว

อ่านโค้ด · FUNCTION · --TASK G1-CURRICULUM-TERRAINRULE

terrain_no_fall: ตัดสินว่าหุ่นตัวไหนขึ้นหรือลงแถว

```
    fell = env.termination_manager.get_term("base_contact")[env_ids]
```

`get_term("base_contact")` หุ่นตัวไหนลำตัวแตะพื้น
`[env_ids]` เลือกเฉพาะหุ่นที่เพิ่งจบ

```
    up = (walked > terrain.cfg.terrain_generator.size[0] / 2) & ~fell
```

`size[0] / 2` แผ่นพื้นกว้าง 8 m จึงต้องเดินเกิน 4 m
`~fell` กลับค่าจริง/เท็จ หมายถึงหุ่นที่ไม่ล้ม
`&` ต้องเป็นจริงทั้งสองข้อ **นี่คือส่วนที่เพิ่มจากกฎเดิม**

```
    terrain.update_env_origins(env_ids, up, fell)
```

`up` หุ่นที่เป็น True ขึ้น 1 แถว
`fell` หุ่นที่ล้มลง 1 แถว **กฎเดิมลงเมื่อเดินไม่ถึงครึ่งของระยะที่สั่ง**

```
    return terrain.terrain_levels.float().mean()
```

`return` แถวเฉลี่ยแสดงในกราฟ Curriculum/terrain_levels

```
self.curriculum.terrain_levels = CurrTerm(func=mdp.terrain_no_fall)
```

`terrain_levels` ใช้ชื่อเดิมใน env_cfg.py เพื่อแทนที่กฎของ G1 Rough

ตัวอย่าง · --TASK G1-CURRICULUM-DIFFICULTY

ใช้ค่าความยากเดียว ควบคุมหลายค่าพร้อมกัน

```python
# 1. นิยามคำว่าสำเร็จ: อยู่รอดจนหมดเวลา
class SurvivedSuccess(ManagerTermBase):
    def __init__(self, cfg, env):
        super().__init__(cfg, env)
        self.succeeded = torch.zeros(env.num_envs, dtype=torch.bool, device=env.device)
    def __call__(self, env):
        self.succeeded[:] = env.termination_manager.time_outs
        return torch.zeros(env.num_envs, device=env.device)

# 2. weight 0 จะไม่ถูกคำนวณ จึงใส่ค่าเล็กมาก
self.rewards.success = RewTerm(func=mdp.SurvivedSuccess, weight=1e-6)

# 3. สำเร็จ +1 ไม่สำเร็จ -1 แยกรายตัว
self.curriculum.difficulty = CurrTerm(func=mdp.DifficultyScheduler,
    params={"init_difficulty": 0, "max_difficulty": 2})

# 4. ค่าที่จะเปลี่ยนตามความยากเฉลี่ย เพิ่มได้หลายค่า
self.curriculum.turn = CurrTerm(func=mdp.modify_term_cfg, params={
    "address": "commands.base_velocity.ranges.ang_vel_z",
    "modify_fn": mdp.initial_final_interpolate_fn,
    "modify_params": {"initial_value": (-.5, .5), "final_value": (-1., 1.),
        "difficulty_term_str": "difficulty"}})
```

iteration	ความยาก
25	0.32
51	0.63
77	0.93
159	**0.997**

ผลจาก run

w_z ถึง ±1.0 และผลักถึง ±1.5 m/s โดยล้มไม่เกินราว 1 % และ Mean reward 23.4

กับดัก: ความยากต่ำกว่า 10 % ระบบยังใช้ค่าจาก config จึงต้องตั้งค่าเริ่มใน config ให้ตรง initial_value

เอาไปใช้

Assignment 7.A และ Challenge 7.5

สูตรทำ 7.A

1. เลือก 1 ค่าจากตารางหน้า 8
2. เริ่มจากค่าเดิม เลือกตามเวลาหรือตามผล
3. เทรน 100–300 it. คู่กับ Base
4. แนบกราฟ Curriculum/<ชื่อ>
5. วัดด้วย eval_g1.py ที่คำสั่งเดียวกัน

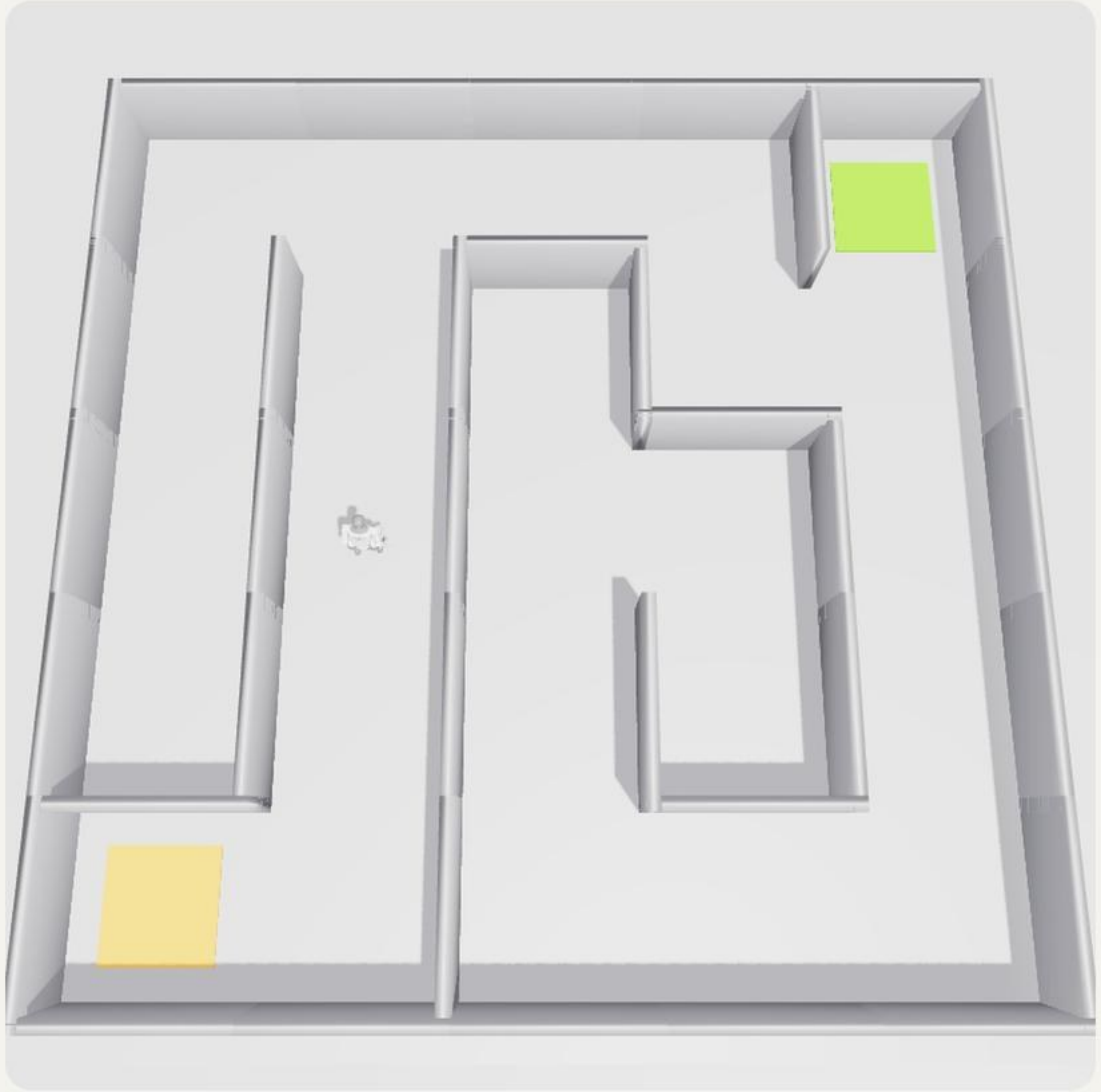
ด่าน Hazard	แนวทางใน G1-Hazard-Starter
ร่อง · เนิน	พื้นไล่ความยากตามแถวอยู่แล้ว ลองใช้ terrain_no_fall ให้ขึ้นแถวยากขึ้น
น้ำแข็ง	friction เป็น event แบบ startup เปลี่ยนระหว่างเทรนไม่ได้ จึงใช้ช่วง 0.2–1.0 ตั้งแต่แรก
ทรงตัวหลังสะดุด	ค่อย ๆ เพิ่ม reset_base หรือ push_robot
ความเร็ว	pilot สั่ง 0.15–0.6 m/s จึงไม่ต้องขยายคำสั่งเกิน 0.8

สถานะ: ตัวอย่างหน้านี้ยังไม่ได้เทรนบน Hazard เป็นแนวทางให้ลองแล้ววัดผลเอง

LAB 7.2 · CHALLENGE

G1 Maze Challenge

รอบนี้ไม่มีคนกดคีย์บอร์ด G1 ต้องใช้ sensor ของตัวเองหาทางจากมุมล่างซ้ายไปมุมบนขวา



G1-Maze-Challenge ใน Newton viewer, กำแพงสูง 1 m, ช่องละ 2 m

CHALLENGE RULES

จาก S ไป G ด้วย sensor ของตัวเอง

แผนที่	5 × 5 ช่อง (10 × 10 m) S อยู่มุมล่างซ้ายหันไป +x, G อยู่มุมบนขวา
สำเร็จ	ลำตัวห่างกลางช่อง G ไม่เกิน 0.6 m ภายใน 150 s
ชน / ล้ม	torso, pelvis, ข้อศอก หรือฝ่ามือแตะอะไรก็ต้องกลับไปเริ่มใหม่ (นับครั้ง)
sensor	ใช้ได้แค่ maze_scanner (height scan) หรือ wall_lidar ห้ามอ่านแผนที่จริงจาก terrain
หุ่น	G1 + policy เดิน G1 Rough (fine-tune ด้วย curriculum ได้)
วัดผล	G1-Maze-Random 16 ตัว ใช้ seed ที่ผู้สอนเลือกวันส่งงาน

SYSTEM

autopilot คือ 3 ชั้นบน policy ที่เดินเป็นแล้ว

maze_scanner
height scan 33 × 33 จุด

→

perception
กำแพง 4 ด้านของช่อง

→

planner
left / right / flood

→

controller
v_x, w_z แบบนุ่ม

→

G1 Rough policy
37 ข้อต่อ · 50 Hz

`autopilot.py`

เปลี่ยนแค่คนสั่ง

เหมือน play_interactive.py ใน Week 3 แต่คำสั่งความเร็วมาจากโค้ดแทนคีย์บอร์ด

`2 height scanners`

คนละหน้าที่

ของ policy 1.6 × 1.0 m ดูพื้นใต้เท้า ของ autopilot 3.2 × 3.2 m ดูกำแพงรอบตัว

`ช่องละ 2 m`

ตัดสินใจทีละช่อง

ถึงกลางช่อง (± 0.35 m) แล้วอ่านกำแพง เลือกช่องถัดไป

แนวคิด: policy เดินไม่รู้จักเขาวงกตเลย เราใช้ **hierarchical control** ให้ชั้นบนวางแผน ชั้นล่าง (RL) ทรงตัวและก้าวเดิน

TERRAINS/MAZE.PY

เขาวงกตคือ sub-terrain ใหม่ใน terrain generator

```
def add_wall(cx: float, cy: float, sx: float, sy: float):
    tf = trimesh.transformations.translation_matrix((cx, cy, 0.5 * h))
    meshes.append(trimesh.creation.box((sx, sy, h), tf))

for i in range(n):
    for j in range(n):
        # west and south walls of every cell + east / north walls on the outer
border
        if walls[i, j, 2]:
            add_wall(x0 + i * c, y0 + (j + 0.5) * c, t, c + t)
        if walls[i, j, 3]:
            add_wall(x0 + (i + 0.5) * c, y0 + j * c, c + t, t)
        if i == n - 1 and walls[i, j, 0]:
            add_wall(x0 + n * c, y0 + (j + 0.5) * c, t, c + t)
        if j == n - 1 and walls[i, j, 1]:
            add_wall(x0 + (i + 0.5) * c, y0 + n * c, c + t, t)

origin = np.array([x0 + 0.5 * c, y0 + 0.5 * c, 0.0])
return meshes, origin
```

trimesh.creation.box

กำแพงคือกล่อง

หนา 0.15 m สูง 1.0 m ช่องละ 2 m (ทางเดินกว้าง 1.85 m)

origin

จุดเกิดอยู่กลางช่อง (0, 0)

เป้าอยู่ช่อง (4, 4) ห่างจากจุดเกิด (8 m, 8 m) ทุกเขาวงกต

difficulty

ใช้ curriculum ได้

difficulty 0 ทุบกำแพงทิ้ง 30 %, 1.0 เป็น perfect maze (ทางเดียว)

TASKS/MAZE/ENV_CFG.PY

วาดเขาวงกตเป็นตัวอักษร หรือสุ่มใหม่ทุกแผ่น

```
# The fixed challenge maze (5 x 5 cells of 2
m). Start S = bottom-left, goal G = top-
right.
# Shortest route: 10 cells. Left-hand wall
following: 18 cells. Right-hand: 30 cells.
CHALLENGE_LAYOUT = """
+-+-+-+-+-+
|       |G|
+ + +-+ + +
| | | |   |
+ + + +-+ +
| | |   | |
+ + + + + +
| | | | | |
+-+ + +-+ +
|S  |     |
+-+-+-+-+-+
"""
```

G1-Maze-Challenge

แผนที่ตายตัว 1 แผ่น

ทางสั้นสุด 10 ช่อง, เกาะซ้าย 18 ช่อง, เกาะขวา 30 ช่อง

G1-Maze-Random

สุ่ม 4 × 4 = 16 แผ่น

generate_maze() ใช้ randomized DFS ทำ perfect maze แล้วทุบกำแพงออกตาม difficulty (แถวบนยากสุด)

G1-Maze-Open

ไม่มีกำแพง

พื้น mesh เปล่า ๆ ไว้ทดสอบ policy เดิน (Exercise 7.3)

SENSOR · RAYCASTERCFG

height scanner 3.2 × 3.2 m รอบตัว

```
cfg.scene.maze_scanner = RayCasterCfg(
    prim_path="{ENV_REGEX_NS}/Robot/torso_link",
    offset=RayCasterCfg.OffsetCfg(pos=(0.0, 0.0, 20.0)),
    ray_alignment="yaw",
    pattern_cfg=patterns.GridPatternCfg(resolution=0.1,
size=[3.2, 3.2]),
    mesh_prim_paths=["/World/ground"],
    global_world_only=True,
    update_period=cfg.decimation * cfg.sim.dt,
    debug_vis=False,
)
```

`offset z = 20 m`

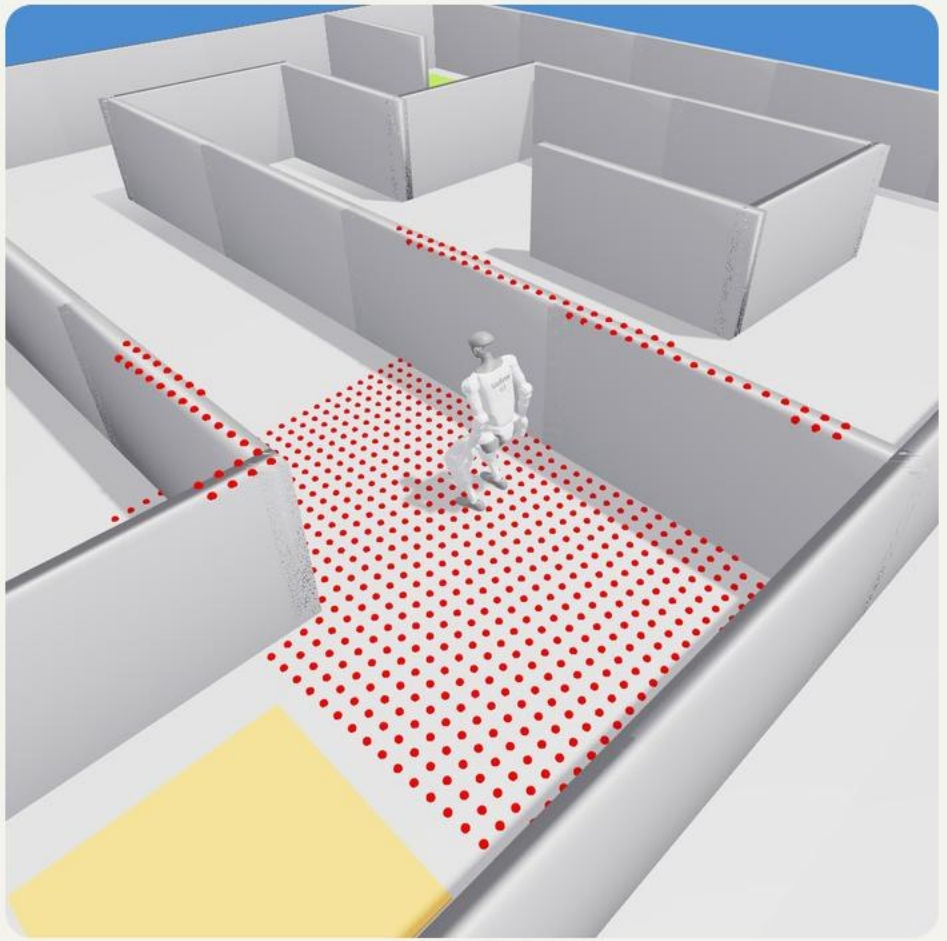
ยิงจากฟ้าลงพื้น

hit z ≈ 0 คือพื้น, ≈ 1.0 คือบนกำแพง

`ray_alignment=yaw`

หมุนตาม yaw อย่างเดียว

หุ่นเอียงตัวเดิน grid ก็ไม่เอียงตาม

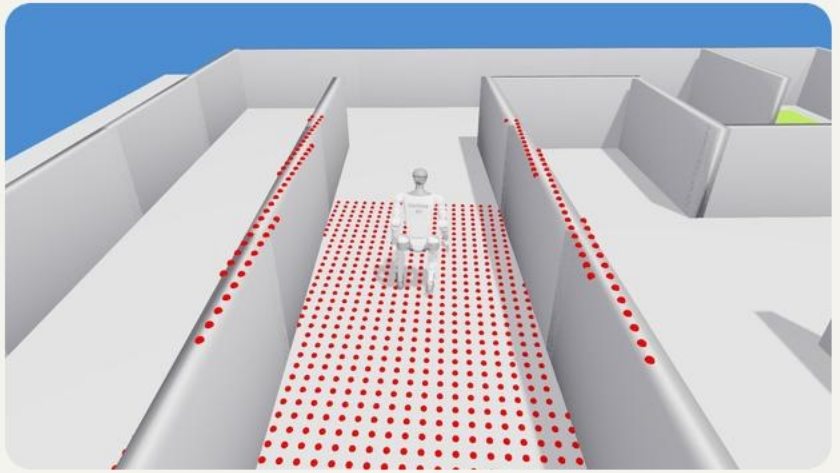


MAZE_SHOW_SCAN=1: จุดที่ rays ชน (debug vis ของ ray caster)

LAYER 1 · PERCEPTION

หากำแพง: นับจุดสูง > 0.5 m ใน “กล่องบาง” แต่ละด้านของช่อง

```
centre = self.centre_of(cell).unsqueeze(1)  # (N, 1, 2)
rel = hits_xy - centre  # (N, R, 2)
tall = hits_z > ground_z + 0.5
out = torch.zeros(hits_xy.shape[0], 4, dtype=torch.bool,
device=self.device)
half = 0.5 * self.c
for d, (dx, dy) in enumerate(DIRS):
    along = rel[..., 0] * dx + rel[..., 1] * dy  # distance towards
side d
    across = rel[..., 0] * dy - rel[..., 1] * dx
    in_box = (torch.abs(along - half) < 0.15) & (torch.abs(across)
< 0.6)
    out[:, d] = (in_box & tall).sum(dim=1) ≥ 3
return out
```



จุดแดง = ray_hits_w ของ scanner: บนพื้นสูง ~0 m, บนสัน กำแพงสูง 1.0 m (> 0.5 m = กำแพง)

กล่องบาง, ≥ 3 จุด

ลึก 0.3 m ยาว 1.2 m

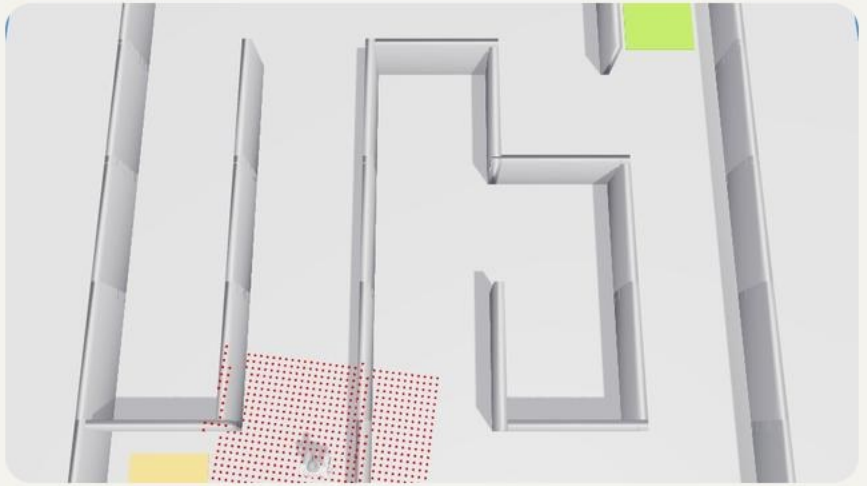
ขอบช่องด้านละกล่อง ต้องมี 3 จุดขึ้นไป กันจุดหลงจากมุมกำแพงข้าง ๆ

LAYER 2 · PLANNER

เลือกช่องถัดไป: เกาะกำแพง หรือ flood fill

```
if self.strategy in ("left", "right"):
    # wall following: try left, straight, right, back (mirror for the right hand)
    turns = (1, 0, 3, 2) if self.strategy == "left" else (3, 0, 1, 2)
    choice = (h + 2) % 4
    for t in reversed(turns):  # the first open side in the preference list wins
        d = (h + t) % 4
        choice = torch.where(open_sides.gather(1, d.unsqueeze(1)).squeeze(1), d,
choice)
    return choice
```

```
dist = self.flood_fill(walls)
idx = torch.arange(len(h), device=self.device)
best = torch.full_like(h, -1)
best_cost = torch.full(h.shape, 10**6, dtype=torch.long, device=self.device)
# straight first, so ties keep the robot walking straight (fewer turns)
for t in (0, 1, 3, 2):
    d = (h + t) % 4
    nb = (cell + self._dirs[d]).clamp(0, self.n - 1)
    cost = dist[idx, nb[:, 0], nb[:, 1]]
    ok = open_sides[idx, d] & (cost < best_cost)
    best = torch.where(ok, d, best)
    best_cost = torch.where(ok, cost, best_cost)
```



มุมบน: scanner เห็นแค่ 3.2 × 3.2 m รอบตัว (จุดแดง) ช่องอื่นต้องจำไว้ใน self.walls: −1 ยังไม่รู้, 0 เปิด, 1 กำแพง

left / right: ลองซ้าย ตรง ขวา แล้วค่อยกลับหลัง ไม่ต้องจำแผนที่

flood: กำแพงที่ยังไม่เห็นถือว่าเปิด เดินไปช่องที่ใกล้เป้าสุด เจอกำแพงใหม่ก็คิดใหม่

LAYER 3 · CONTROLLER

เดินไปกลางช่องถัดไป ชะลอตอนเลี้ยวแต่ไม่หมุนอยู่กับที่

```
to_target = self.centre_of(self.target) - pos_xy
yaw_err = torch.atan2(to_target[:, 1], to_target[:, 0]) - yaw
yaw_err = torch.atan2(torch.sin(yaw_err), torch.cos(yaw_err))
# full speed when facing the target, down to 0.15 m/s when it is 90 deg or more
to the side
v_des = 0.15 + (self.speed - 0.15) * torch.clamp(torch.cos(yaw_err), min=0.0) **
2
w_des = torch.clamp(1.5 * yaw_err, -self.max_turn, self.max_turn)
# rate limits [m/s^2], [rad/s^2]
self.cmd[:, 0] += torch.clamp(v_des - self.cmd[:, 0], -self.max_acc * dt,
self.max_acc * dt)
self.cmd[:, 2] += torch.clamp(w_des - self.cmd[:, 2], -self.max_ang_acc * dt,
self.max_ang_acc * dt)
return self.cmd.clone()
```

cos²(yaw error)

หันตรงก็เต็มสปีด

หันผิด 90° ขึ้นไปจะเหลือ 0.15 m/s แต่ยังก้าวอยู่ (ไม่หมุนอยู่กับที่) $w_z \le 0.8$ rad/s ตามช่วงที่ G1 Rough เทรนมา

rate limit

0.8 m/s² · 2 rad/s²

ปิดได้ด้วย MAZE_ACC=1e9,1e9 วัดแล้วก็ไม่ล้มเหมือนกัน

ผลวัด: ตอนแรกคิดว่าคำสั่งกระตุกทำให้ล้ม เลยใส่ rate limit แต่ปิดแล้วก็ **ล้ม 0 ครั้ง** (flood 39.4 s vs 41.3 s, left 73.3 vs 75.7 s, 16 ตัว) ตัวการจริงคือ policy Flat บนพื้น mesh (หน้า “สิ่งที่เจอระหว่างทำ”) ตั้งสมมติฐานแล้วต้องวัดเสมอ

RUN

ดูหุ่นเดินเอง แล้ววัดผลแบบ headless

```
$ uv run python scripts/maze_autopilot.py --task G1-Maze-Challenge --num_envs 1 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt --viz newton
# MAZE_STRATEGY=left|right|flood  MAZE_SENSOR=height|lidar  MAZE_SPEED=0.6  MAZE_CAM=chase|top
[MAZE] t= 10.0s robot0 cell=[1, 1] cells walked=2
[MAZE] robot 0: GOAL at 41.2 s after 10 cells
[MAZE] map seen by robot 0 ('?' = never sensed):
```

```
+-+-+-+-+-+
|       |G|
+?+ +-+ + +
| | | |   |
+?+ +?+-+ +
| | | ? ? |
+?+ +?+?+?+
| | | ? ? |
+-+ +?+?+?+
|S  | ? ? |
+-+-+-+-+-+
```

`flood, 10 ช่อง`

เห็นกำแพงแค่ที่ผ่าน

'?' คือด้านที่หุ่นไม่เคยยืนดู flood fill ถือว่าเปิด รอบนี้โชคดีที่ทางนั้นถูก

`--num_envs 16 ไม่ใส่ --viz`

เร็วกว่าเวลาจริง ~2×

150 s ในซิม ≈ 80 s บนโน้ตบุ๊ก ได้สถิติ 16 ตัวในรอบเดียว

ผล LAB 7.2: PRETRAINED G1 ROUGH, 0.6 M/S, 16 ตัว

ทุก strategy ถึงเป้า 16/16 โดยไม่ล้มเลย

strategy · sensor	ถึงเป้า	เวลาเฉลี่ย	ช่วง	ช่องที่เดิน	ชน/ล้ม
left · height	16/16	75.7 s	74.3 – 76.9	18	0
right · height	16/16	130.9 s	128.6 – 134.2	30	0
flood · height	16/16	41.3 s	40.6 – 42.6	10	0
left · lidar	16/16	75.4 s	73.9 – 76.8	18	0
left · 16 random	16/16	77.9 s	30.3 – 104.7	17.9	0
flood · 16 random	16/16	49.0 s	30.0 – 73.9	11.8	0

flood

เร็วสุด

41 s เดินตามทางสั้นสุด 10 ช่องพอดีในแผนที่ challenge

left vs right

ขึ้นกับแผนที่

แผนที่นี้ซ้ายได้เปรียบ (18 vs 30 ช่อง) แต่บางแผนที่กลับกัน

16 random

flood ยังชนะ

เฉลี่ย 11.8 ช่อง vs 17.9 ช่อง (left)

headless, ช่องละ 2 m, เวลารวมตอนเลี้ยว, ถึงเป้าที่ ≤ 0.6 m

ลองเร่งเป็น 1.0 M/S (FLOOD, 16 RANDOM)

เร็วขึ้น ≠ คะแนนดีขึ้น

policy เดิน	MAZE_SPEED	ถึงเป้า	เวลาเฉลี่ย	ชน/ล้ม	score*
pretrained G1 Rough	0.6	16/16	49.0 s	0	51.0
pretrained G1 Rough	1.0	15/16	43.2 s	21	−54.5
Velocity curriculum (fix)	1.0	15/16	45.0 s	18	−41.3
Full curriculum (fix)	1.0	14/16	41.6 s	21	−59.1

1.0 m/s

เร็วขึ้น ~6 s แต่ล้ม 18–21 ครั้ง

ทางตรงไม่น่าใช่สาเหตุ: eval สั่งเดินตรง 2.0 m/s ล้ม 0 ทุก policy เลยสงสัยช่วงเลี้ยวมุม (ยังไม่ได้พิสูจน์, MAZE_DEBUG=1 + viewer)

curriculum 300 it.

ไม่ช่วยในโจทย์นี้

policy ที่ fine-tune ด้วย Velocity / Full ยังล้มพอ ๆ กับ pretrained ที่ 1.0 m/s

โจทย์ challenge

ตรงนี้ยังเก็บคะแนนได้

ชะลอก่อนเลี้ยว เร่งในทางตรง หรือเทรน curriculum เรื่องการเลี้ยว (w_z)

* score = 100 × ถึงเป้า/16 − เวลาเฉลี่ย − 5 × ชน/ล้ม (หน้า challenge)

สิ่งที่เจอระหว่างทำ

policy Flat ของ Week 3 ล้มบนพื้นเขาวงกต แต่ Rough ไม่ล้ม

policy	พื้น	0.4 m/s + 0.8 rad/s	0.6 m/s + 1.0 rad/s	0.15 m/s + 0.8 rad/s
G1 Flat (Week 3)	plane (Isaac-Velocity-Flat-G1)	0	0	0
G1 Flat (Week 3)	mesh (G1-Maze-Open-FlatPolicy)	19	17	0
G1 Rough	mesh (G1-Maze-Open)	0	0	0

ล้มต่อ 64 ตัว ใน 15 s

พื้นเรียบเหมือนกัน แต่คนละชนิด

plane คือระนาบคณิตศาสตร์ terrain generator คือกล่อง mesh contact เลยต่างกัน

Flat ทั้งเขาวงกต

15/16 ถึงเป้า ล้ม 24 ครั้ง

flood 16 ตัว ตั้งค่าเหมือนกับที่ Rough ได้ 16/16 ล้ม 0

Rough

ผ่าน terrain curriculum

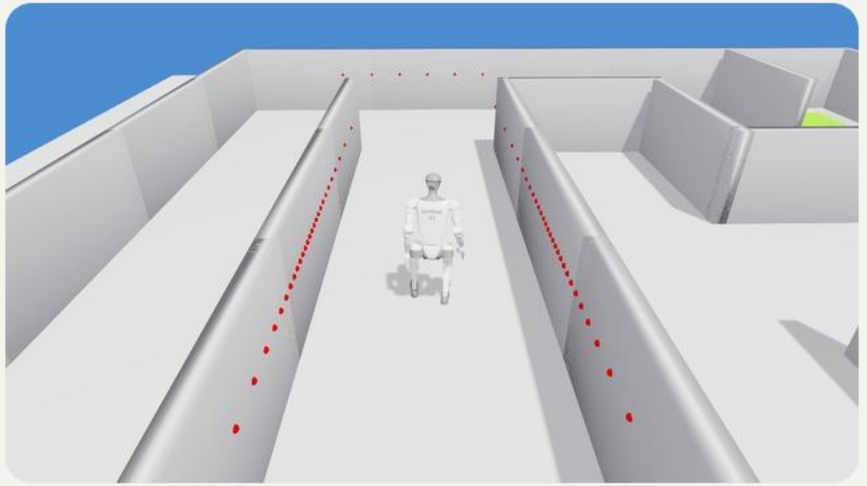
เทรนบน mesh หลายระดับความยากมาเลยทนกว่า จึงใช้เป็น policy เดินของ challenge

บทเรียน: domain gap เล็ก ๆ (plane กับ mesh) ก็ทำ policy พังได้ วิธีกันคือใช้ curriculum พาไปเจอสภาพหลาย ๆ แบบระหว่างเทรน

SENSOR ที่สอง: WALL_LIDAR

lidar 2D: 72 rays แนวนอน ทุก 5° ที่ระดับอก

```
# ray_alignment="yaw" rotates only the ray start points (it was
made for height scanners),
# so we take each ray's world angle from its hit point; rays that
hit nothing are ignored
rel = lidar.data.ray_hits_w.torch[..., :2] -
lidar.data.pos_w.torch[:, None, :2]
dist = torch.nan_to_num(torch.linalg.norm(rel, dim=-1), nan=1e6,
posinf=1e6)
angles_w = torch.atan2(rel[..., 1], rel[..., 0])
walls_now = pilot.walls_from_lidar(dist, angles_w, pos_xy -
pilot.centre_of(cell))
```



จุดแดง = ที่ lidar ยิงโดน เป็นเส้นบนผนังระดับอก ไม่เห็นพื้น

bug ที่เจอ → แก้

ทิศ rays ไม่หมุนตามหุ่น

บวก yaw ซ้ำ กำแพงเพี้ยน 90° ได้ 0/16 → คิดมุมจากจุดที่ชน ได้ 16/16

CHALLENGE: ส่งงาน

ทำให้ G1 ออกจากเขาวงกตเร็วที่สุด

แก้ได้

autopilot.py ทั้ง 3 ชั้น

perception (sensor ไหน กี่จุด), planner (strategy ใหม่), controller (สปีด การเลี้ยว)

แก้ได้

policy เดิน

fine-tune G1 Rough ด้วย curriculum ของ Lab 7.1 ให้เดินเร็ว/ทนขึ้น

ห้าม

อ่านแผนที่จริง

ต้องรู้กำแพงจาก sensor เท่านั้น ห้ามแก้ termination / ขนาดเขาวงกต

```
$ MAZE_STRATEGY=flood uv run python scripts/maze_autopilot.py --task G1-Maze-Random --num_envs 16 \
    --checkpoint <policy ของคุณ> env.scene.terrain.terrain_generator.seed=<seed วันส่งงาน>
[MAZE] RESULT strategy=flood sensor=height: 16/16 reached the goal, time mean 49.0 s ... crashes/falls 0
```

Score: คะแนน = 100 × (จำนวนที่ถึงเป้า / 16) − เวลาเฉลี่ย (s) − 5 × จำนวนครั้งที่ชน/ล้ม, baseline (flood, 0.6 m/s) = 100 − 49.0 − 0 = **51.0**

EXERCISES · WEEK 7

ลงมือทำ

มี 4 โจทย์ ทุกข้อรันบนโน้ตบุ๊กได้ เฉลย (พื้นเขียว) อยู่ต่อจากแต่ละข้อ

EXERCISE 7.1 · TUNE THE RAMP

ramp feet_air_time ให้สูงขึ้น เดินต่างไปไหม

7.1

```
$ uv run isaaclab train --rl_library rsl_rl --task G1-Curriculum-Reward --num_envs 1024 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=300 \
    agent.run_name=reward_w2 env.curriculum.feet_air_time_ramp.params.end_weight=2.0
```

โจทย์

1. ramp นี้เริ่มที่ iteration ไหนใน log และถึงค่าสุดท้ายที่ iteration ไหน? (log เริ่ม 5000, start_step = 50 × 24)
2. วัด air time ต่อก้าว (eval_g1.py, สั่ง 0.8 m/s) ของ Base, Reward (1.0) และ Reward (2.0) เปลี่ยนไปทางไหน?
3. ทำไม ramp ถึงดีกว่าเปลี่ยน weight ทีเดียว (modify_reward_weight) ดูกราฟ Mean reward ตอนเปลี่ยน

เฉลย EXERCISE 7.1

end_weight 2.0 ก้าวนานขึ้น ~10 % แต่เดินช้าลงนิดหน่อย

1. start_step = 50 × 24 = 1,200 step เริ่มขึ้นที่ iteration **5050** และถึงค่าสุดท้ายที่ **5150** (150 × 24) log เริ่ม 5000 แต่ common_step_counter เริ่ม 0
2. air time ต่อก้าวที่ 0.8 m/s: Base 0.119 s, end 1.0 ได้ 0.122 s, end 2.0 ได้ **0.132 s** (Episode_Reward/feet_air_time 0.032 → 0.075) แต่ v_x ที่สั่ง 2.0 ลดจาก 1.846 เหลือ 1.770 m/s
3. modify_reward_weight เปลี่ยนทีเดียว reward scale กระโดด critic เลยประเมินค่าผิดไปพักหนึ่ง ramp ค่อย ๆ เปลี่ยนให้ critic ตามทัน (สไลด์นี้ไม่ได้รันแบบกระโดดมาเทียบ ลองเป็นการบ้านเสริม)

run	weight	air (s)	v_x @2.0
Base	0.25	0.119	1.846
Reward · end 1.0	1.0	0.122	1.829
Reward · end 2.0	2.0	0.132	1.770

eval_g1.py, 64 ตัว, rough terrain

weight ที่ใหญ่ขึ้น คือการแลกความเร็วกับท่าก้าวยาว ramp แค่ช่วยให้เปลี่ยนแบบนุ่ม ๆ ไม่ได้เปลี่ยนว่าสุดท้ายแลกอะไร

EXERCISE 7.2 · YOUR OWN TERM

เขียน curriculum ด้วย modify_term_cfg (ไม่ต้องสร้าง class)

7.2

```
CurrTerm(func=mdp.modify_term_cfg,
    params={"address": "rewards.action_rate_l2.weight",
            "modify_fn": ???, "modify_params": {...}})
```

โจทย์

1. เขียน modify_fn(env, env_ids, old_value, **params) ให้ action_rate_l2 weight ไล่จาก −0.005 → −0.02 ใน 80 iterations
2. เพิ่มเป็น task ใหม่ G1-Curriculum-ActionRate แล้ว fine-tune 100 it. กราฟ Curriculum/... ขึ้นตามที่ตั้งไหม?
3. ถ้า modify_fn คืน mdp.modify_term_cfg.NO_CHANGE จะเกิดอะไร และควรใช้เมื่อไร?

เฉลย EXERCISE 7.2

modify_term_cfg ทำงานได้ แต่ไม่ขึ้นกราฟ Curriculum

1. ดูโค้ดด้านขวา (อยู่ใน tasks/curriculum/env_cfg.py) ลองรันแล้ว G1-Curriculum-ActionRate 100 it. ≈ 2 นาที

2. modify_term_cfg คืนค่า **None** rsl_rl เลยไม่ log Curriculum/action_rate_weight ให้ ต้องเช็คจาก Episode_Reward/action_rate_l2: −0.49 (it. 5025) → −1.07 → −1.55 → **−1.85** (5098) เทียบ Base ≈ −0.51

3. NO_CHANGE คือไม่เรียก setter เลย (ไม่เขียนค่าซ้ำ) ใช้ตอนยังไม่ถึงเงื่อนไข เช่น ก่อน num_steps จะคืน None ไม่ได้ เพราะ None อาจเป็นค่าที่ตั้งใจเขียน

```python
def action_rate_schedule(env, env_ids, old_value, start_step: int,
end_step: int, start: float, end: float):
    frac = min(max((env.common_step_counter - start_step) /
max(end_step - start_step, 1), 0.0), 1.0)
    return start + frac * (end - start)

self.curriculum.action_rate_weight = CurrTerm(
    func=mdp.modify_term_cfg,
    params={
        "address": "rewards.action_rate_l2.weight",
        "modify_fn": action_rate_schedule,
        "modify_params": {
            "start_step": 0,
            "end_step": 80 * STEPS_PER_IT,
            "start": -0.005,
            "end": -0.02,
        },
    },
)
```

ถ้าอยากเห็นกราฟ ให้ term return ตัวเลข จะเป็น function หรือ class ก็ได้ หรือใช้ Ramp จากหน้าตัวอย่างเพิ่มเติม

EXERCISE 7.3 · DOMAIN GAP

policy เดียวกัน พื้นก็เรียบเหมือนกัน ทำไมล้ม? 7.3

```
$ EVAL_STEPS=750 EVAL_CMDS="0.4,0,0.8;0.6,0,1.0" uv run python scripts/eval_g1.py --num_envs 64 \
    --task Isaac-Velocity-Flat-G1 | G1-Maze-Open-FlatPolicy --checkpoint pretrained/g1_flat/model_pretrained.pt
$ ... --task G1-Maze-Open --checkpoint pretrained/g1_rough/model_pretrained.pt
```

โจทย์

1. รันทั้ง 3 แบบ (สั่งเดินเป็นวงกลม ไม่ชนขอบ) นับจำนวนล้มต่อ 64 ตัว ใน 15 s
2. พื้น 2 แบบต่างกันตรงไหน (ดู terrain_type ใน G1FlatEnvCfg กับ maze_terrain_cfg)? ทำไม policy Rough ถึงไม่ล้ม?
3. ถ้าต้องใช้ policy Flat ในเขาวงกตจริง ๆ จะแก้ที่ environment หรือที่การเทรน? คิดมา 1 วิธีที่ใช้ curriculum

เฉลย EXERCISE 7.3

plane กับ mesh คือคนละพื้นสำหรับ policy

1. ล้มต่อ 64 ตัว ใน 15 s (0.4+0.8 / 0.6+1.0 / 0.15+0.8): Flat บน plane **0 / 0 / 0**, Flat บน mesh **19 / 17 / 0**, Rough บน mesh **0 / 0 / 0**
2. G1FlatEnvCfg ใช้ terrain_type="plane" (ระนาบไม่มีที่สิ้นสุด contact แบบ analytic) เขาวงกตใช้กล่อง mesh จาก terrain generator contact ต่างกันนิดเดียวแต่ก็พอให้ policy ที่ไม่เคยเจอล้ม ส่วน Rough เทรนบน mesh หลายระดับผ่าน terrain_levels
3. แก้ที่การเทรน: fine-tune Flat บน terrain generator เริ่มจากแถวเรียบ (difficulty 0) แล้วให้ terrain_levels พาไปพื้นยาก (นี่คือ curriculum) หรือสุ่มแรงเสียดทาน/พื้นระหว่างเทรน

domain gap เล็ก ๆ ก็ทำ policy พังได้ เลยควรเทรนบนสภาพที่หลากหลายกว่าตอนใช้จริงไว้ก่อน

EXERCISE 7.4 · SENSOR DESIGN

scanner หยาบลงหรือเล็กลง autopilot ยังเห็นกำแพงไหม

7.4

```
$ MAZE_STRATEGY=flood uv run python scripts/maze_autopilot.py --task G1-Maze-Challenge --num_envs 16 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt env.scene.maze_scanner.pattern_cfg.resolution=0.2
# ลองต่อ: resolution=0.15 · pattern_cfg.size=[2.0,2.0] · MAZE_SENSOR=lidar
```

โจทย์

1. กำแพงหนา 0.15 m ถ้า resolution 0.1 / 0.15 / 0.2 m จะมี ray ตกบนสันกำแพงกี่แถว? เดาผลก่อนรัน
2. รันทั้ง 3 ค่า + size 2.0 m จดผลถึงเป้า / เวลา / ชน แล้วอธิบายตัวที่พัง
3. แก้ walls_from_height_scan ให้ทนกับ scan หยาบขึ้น (เช่น ขยายกล่อง หรือลดจำนวนจุดขั้นต่ำ) แล้วรันใหม่

เฉลย EXERCISE 7.4

resolution หยาบลงยังรอด แต่ scanner เล็กลงพังทันที

1. rays ยิงดิ่งลง เลยโดนกำแพงแค่ที่สันบน (หนา 0.15 m): 0.1 ได้ 1–2 แถว, 0.15 ได้ 1 แถว, 0.2 ได้ 0–1 แถว (อาจพลาด) แต่แถวเดียวยาว 1.2 m มี 6–12 จุด ≥ 3 จุดก็ผ่าน

2. flood, challenge, 16 ตัว: 0.1 → 16/16, 41.3 s, 0 ชน / 0.15 → 16/16, 41.3 s, 0 / 0.2 → 16/16, 41.4 s, 0 / **size 2.0 m → 0/16, ชน 439 ครั้ง** เพราะ grid ±1.0 m แต่กล่องตรวจกำแพงอยู่ที่ 0.85–1.15 m จากกลางช่อง หุ่นยืนเยื้องนิดเดียวก็มองไม่เห็นกำแพงฝั่งไกล

3. ตัวอย่างการแก้: ขยับกล่องเข้ามา (0.85–1.0 m) ลด min hits เป็น 1 สำหรับ scan เล็ก หรือใช้ lidar (ระยะ 6 m) ลองเองแล้วรายงานผล

ออกแบบ sensor จากสิ่งที่ต้องเห็น ขนาดต้องคลุมกำแพงทั้ง 4 ด้านรวมระยะเยื้อง และความละเอียดต้องเล็กกว่าความหนาของวัตถุ

ASSIGNMENT 7

ส่ง 2 ส่วน

7.A: Curriculum 1 ฟังก์ชัน

เขียน curriculum term ใหม่ 1 ตัว (function หรือ class) อธิบายว่าปรับอะไร ใช้เงื่อนไขอะไรเพิ่ม/ลดความยาก (เช่น “ถึง iteration 150 ของรอบนี้ เพิ่ม feet_air_time”) แนบกราฟ Curriculum/<ชื่อ> จาก TensorBoard

7.B · G1 Maze Challenge

autopilot.py ที่แก้แล้ว + checkpoint (ถ้า fine-tune) + บรรทัด [MAZE] RESULT บน G1-Maze-Random 16 ตัว + อธิบาย 1 หน้า แก้อะไร ทำไมเร็วขึ้น/ชนน้อยลง

ไฟล์: โปรเจกต์ + checkpoint ทั้งหมดอยู่ใน Week07_Curriculum_Maze/g1_maze_lab ส่วนเฉลยโค้ดอยู่ในสไลด์พื้นเขียว

ต่อจากนี้ · CHALLENGE 7.5

G1 Hazard Maze: แข่งจับเวลาบนจอใหญ่

ใช้ทุกอย่างจาก Lab 7.1 กับ 7.2 ในเขาวงกตที่มีร่อง เนิน น้ำแข็ง และหนามที่แตะแล้วล้ม ซ้อม 7 × 7 กับ 9 × 9 แข่งจริง 10 × 10 สุ่มจุดเริ่มและเส้นชัย

G1-Hazard-Starter

ฝึกขา

task ตั้งต้นที่เทรนได้เลย ก๊อปไปแก้พื้นฝึกให้เหมือนด่านมากขึ้น ใช้ curriculum ของ Lab 7.1

pilot.py

เขียนสมอง

ต่อยอด autopilot ของ Lab 7.2 ให้ดูทางจาก height scan แล้วเลือกทางที่หุ่นไปไหว

http://<IP ครู>:8000

ส่งแล้วดูผล

มีรอบซ้อมที่รันให้ทันทีหลังส่ง แล้วค่อยแข่งจริงด้วยด่านและ seed ที่ครูประกาศ

ใบโจทย์เต็ม: Challenge 7.5 — G1 Hazard Maze